virtusa
Accelerating Business Outcomes

Architecture Document for *Autonomous Fraud Detection & Response Network*

Multi AI Agent Fraud Detection System – Phase 2 MVP Version

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 PURPOSE 1
1.2 SCOPE 2
1.3 DEFINITIONS, ACRONYMS AND ABBREVIATIONS 3
1.4 REFERENCES 4

2 ARCHITECTURAL GOALS AND CONSTRAINTS 6

2.1 REUSABILITY 6
2.2 SCALABILITY 6
2.3 CUSTOMIZABILITY 7
2.4 EXTENDIBILITY 8
2.5 USE OF EXISTING BUSINESS LOGIC 8
2.6 TIME TO MARKET 9
2.7 PORTABILITY 10
2.8 AVAILABILITY 10
2.9 PERFORMANCE 11
2.10 ANY OTHER CRITICAL GOALS APPLICABLE 12

3 PRODUCTIZATION ASSESSMENT 13

3.1 RE-USABLE COMPONENTS 13
3.2 ANALYZE ARCHITECTURAL FRAMEWORKS IN REPOSITORY 13
3.3 IDENTIFY AND ANALYZE OPEN SOURCE AND COTS PRODUCTS 14

4 SYSTEM ARCHITECTURE 15

4.1 OVERVIEW 15
4.2 LOGICAL/FUNCTIONAL VIEW 16
4.3 USE CASE VIEW 17
4.4 IMPLEMENTATION/SYSTEM VIEW 18
4.5 PROCESS/THREAD VIEW 19
4.6 DEPLOYMENT VIEW 19

5 GENERAL ARCHITECTURE FOR CORE TECHNICAL SERVICES 21

5.1 PERSISTENCE 21
5.2 INTER-PROCESS COMMUNICATION 23
5.3 AUTHENTICATION AND AUTHORIZATION 26
5.4 ERROR HANDLING 29
5.5 LOGGING 31
5.6 TRANSACTION MANAGEMENT 34
5.7 OTHER APPLICABLE TECHNICAL SERVICES 36

6 RISKS/LIMITATION 39

7 ALTERNATIVE SOLUTIONS CONSIDERED 40

7.1 MAJOR ARCHITECTURAL ALTERNATIVE 40
7.2 EVOLUTION PATHS 43

APPENDIX 44

1. EXPECTED SOFTWARE RESPONSE 44
2. PERFORMANCE BOUNDS 45
3. IDENTIFICATION OF CRITICAL COMPONENTS 46
4. REVIEW COMMENTS ON ARCHITECTURAL POC 47
5. JUSTIFICATION OF CHANGES TO EXISTING ARCHITECTURE (OPTIONAL) 48

1 Introduction

This document presents a comprehensive architectural overview of the **Eagle Trust Bank — Autonomous Fraud Detection & Response Network** system, utilizing various architectural perspectives to illustrate different facets of the system. It aims to encapsulate and communicate the significant architectural decisions made regarding the system. This is a multi-agent fraud detection and prevention platform designed to identify, evaluate, and mitigate fraudulent financial transactions in real-time. The system employs a distributed agent-based architecture comprising four specialized autonomous agents: the Pattern Recognition Agent (PRA), Risk Assessment Agent (RAA), Alert & Block Agent (ABA), and Compliance Logging Agent (CLA).

These agents operate within a coordinated polling pipeline. Raw transactions are processed sequentially through this pipeline, where each agent applies machine learning anomaly detection, risk scoring, regulatory evaluation, and compliance artifact generation (such as STRs, CTRs, and case filings) before archiving to audit trails. The platform integrates a Flask-based backend microservice architecture with a React frontend, supported by MySQL for transactional persistence and ChromaDB for vector-based knowledge retrieval of regulatory frameworks and fraud typologies. Key architectural decisions include asynchronous background-threaded agent pollers (operating at 500ms intervals) to enable non-blocking transaction processing, BERT embeddings for RAG-augmented decision-making, multi-factor verification via OTP challenges for high-risk transactions, and comprehensive audit trail preservation for forensic analysis and regulatory compliance.

1.1 Purpose

This document provides a comprehensive architectural blueprint of the **Eagle Trust Bank — Autonomous Fraud Detection & Response Network**, emphasizing the critical design decisions that shape its structure and implementation. It employs a multi-view architecture approach to encapsulate various perspectives of the system, which include:

- **Logical View:** Depicts the four autonomous agents (PRA, RAA, ABA, CLA), supporting services (Payment, Account, Authentication, OTP), and data stores (MySQL transactional, ChromaDB vector), illustrating the interaction of components through a coordinated polling pipeline.
- **Use Case View:** Aligns core functionalities (fraud detection, risk assessment, remedial action, regulatory compliance) with architectural components and delineates component responsibilities within the transaction processing workflow. Implementation View: Elaborates on sub-modules within each agent package, key Python classes (PRA Agent, RAA Agent, ABA Agent, CLA Agent), ML model instantiation, DAO layers, and code organization across `et_service/`, `et_api/`, `et_dao/`, and `et_model/` directories.

- **Process View:** Describes workflows including transaction ingestion → TMA synchronous analysis → PRA asynchronous pattern detection → RAA risk scoring and STR/CTR determination → ABA action execution → CLA document generation and archival; it also details the RAG pipeline through five-tier ChromaDB collections and BERT embedding generation.
- **Deployment View:** Outlines the Flask microservice deployment topology, MySQL database connectivity, ChromaDB persistence directories, model artifact locations (BiLSTM, Isolation Forest), and the background polling thread architecture facilitating non-blocking agent execution.

This architectural guide is intended for all stakeholders—such as developers, architects, compliance managers, and bank executives—to provide a clear understanding of the system's construction and its support for business objectives like minimizing fraud-related losses, streamlining regulatory compliance (AML/KYC), reducing investigation turnaround time, and enhancing operational efficiency. It elucidates how artificial intelligence, machine learning, and distributed agent systems are seamlessly integrated to enable intelligent, real-time fraud detection and regulatory response for critical banking transactions, customer accounts, and compliance filings. The system's knowledge-augmented architecture combines historical fraud patterns, regulatory frameworks, and behavioural profiling to facilitate intelligent risk scoring and justified remedial actions while maintaining comprehensive audit trails for forensic analysis and regulatory validation.

1.2 Scope

This document outlines the architecture of the Autonomous Fraud Detection System, covering core components and workflows.

➢ IN SCOPE:

- **Agent Architecture:** Multi-agent system (TMA, PRA, RAA, ABA, CLA) handling real-time detection, pattern analysis, risk scoring, response actions and compliance documentation.
- **Machine Learning Component:** Uses Isolation Forest, BiLSTM and Sentence Transformer with feature engineering for anomaly detection and behavioral analysis.
- **Retrieval-Augmented Generation (RAG) System:** Five-layer ChromaDB vector store enabling similarity search and AI-driven risk justification using regulatory and historical data.
- **Data Architecture:** MySQL schema with agent state flags, ChromaDB persistence, and audit trails for secure and traceable data hanfling.
- **Authentication & Authorization:** OTP-based MFA, secure sessions, role-based access and admin/customer portals with bcrypt password protection.

- **API Layer:** Flask REST APIs supporting transactions, alerts, account management, agent queries, enabled frontend integration.
- **Compliance & Regulatory Components:** STR\CTR generation, legal citation tracking, PDF reports via ReportLab and case filing management.
- **Frontend Integration:** React dashboard for fraud monitoring, alert updates, transaction insights and PDF export functionality.

➢ OUT OF SCOPE:
- UI/UX design details, infrastructure setup, and deployment strategies
- External payment gateway integration and legacy data migration
- Training materials, QA plans, and network/security architecture
- Data privacy policies, notification systems, and performance testing.
- Disaster recovery and business continuity planning

1.3 Definitions, Acronyms and Abbreviations

Definitions, Acronyms and Abbreviations	
TMA	Transaction Monitoring Agent – Stage 1 of the Agentic pipeline
PRA	Pattern Recognition Agent – Stage 2, BiLSTM sequence analysis
RAA	Risk Assessment Agent – Stage 3, multi-dimensional scoring
ABA	Alert and Block Agent – Stage 4, payment action executor
CLA	Compliance Logger Agent – Stage 5, PMLA archival
RAG	Retrieval – Augmented Generation – ChromaDB knowledge retrieval
ML	Machine Learning – Isolation Forest anomaly detection
BiLSTM	Bidirectional Long Short-Term Memory neural network
PMLA	Prevention of Money Laundering Act(India)
FIU-IND	Financial Intelligent Unit - India
CTR	Cash Transaction Report (PMLA mandatory reporting)
STR	Suspicious Transaction Report (PMLA mandatory reporting)
OTP	One-Time Password – fraud multi-factor authentication
L1/L2/L3	ChromaDB collection tiers: Regulatory/ cases/ Typoligies
HNSW	Hierarchial Navigate Small World – ChromaDB index algorithm
JWT	JSON Web Token – authentication mechanism
MFA	Multi-Factor Authentication
T1 – T4	Customer tier classification (T1 = highest risk, T4 = veteran)

1.4 References

1. Reserve Bank of India (RBI) Master Circular on AML/KYC & STR Standards Framework for anti-money laundering, know-your-customer compliance, and suspicious transaction reporting guidelines that govern fraud detection and response protocols in Indian banking systems. https://rbidocs.rbi.org.in/

2. IEEE Std 1471-2000: Architectural Description of Software-Intensive Systems Industry-standard specification for documenting multi-agent system architectures, component interactions, deployment topology, and design rationale for distributed fraud detection platforms. https://standards.ieee.org/

3. Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" Facebook AI Research (2020) Foundational methodology for the RAG system (L1-L5 ChromaDB tiers) enabling knowledge-augmented decision-making in risk assessment and regulatory compliance determination. https://arxiv.org/abs/2005.11401

4. Liu et al. "Isolation Forest" — ACM TKDD (2012) Core machine learning algorithm for unsupervised anomaly detection in fraud pattern recognition agent, detecting behavioural outliers in transaction data without labelled examples. https://dl.acm.org/doi/10.1145/2133360.2133363

5. Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" — ACL (2019) Basis for Sentence Transformer (all-MiniLM-L6-v2) embeddings used in semantic similarity search across regulatory frameworks, fraud typologies, and case precedents in ChromaDB. https://arxiv.org/abs/1810.04805

6. Wooldridge, M. "An Introduction to Multiagent Systems" (2nd ed.) — Wiley (2009) Foundational reference for multi-agent architecture design, agent autonomy, communication patterns (database polling), and coordination mechanisms in the PRA-RAA-ABA-CLA pipeline. https://www.wiley.com/

7. NIST SP 800-63B: Digital Identity Guidelines — Authentication and Lifecycle Management framework for implementing secure authentication mechanisms including OTP-based multi-factor verification, session management, and password security in the fraud detection system. https://pages.nist.gov/800-63-3/sp800-63b.html

2 Architectural Goals and Constraints

The Autonomous fraud detection system is designed for scalability, reusability, and rapid deployment through a modular agent-based architecture and configuration-driven design. It targets growth from 20 to 100+ TPS with sub-500ms latency, while enabling customization via pluggable models and standard interfaces. A 3-month MVP is achieved using a monolithic setup with pre-built regulatory datasets, with Phase 3 focusing on portability, high availability (99.95%), and full PCI-DSS and AML/KYC compliance.

2.1 Reusability

Goal: Enable fraud detection components to be reused across multiple deployment contexts and reduce development rework by leveraging existing business logic and proven patterns.

Requirements: The system must support integration into multiple deployment scenarios including (1) Phase 2 RDBMS migration from legacy systems, (2) multi-tenant SaaS offerings for third-party banks, and (3) integration as microservices via third-party APIs. Agents must operate independently without tight coupling to Flask or any specific framework.

Current Status: 85% completion. Clean Data Access Object (DAO) abstraction layer isolates SQL logic from business logic (et_dao/ modules). Each agent module (monitoring_agent/, pattern_agent/, raa/) encapsulates independent business logic with no direct agent-to-agent calls. Domain models use Python @dataclass decorators without ORM framework dependencies, enabling serialization to any format (JSON, Protocol Buffers, etc.).

Identified Gaps:

- No library packaging mechanism (monolithic application structure)
- No semantic versioning for agent components
- Database-specific type definitions hardcoded
- Framework coupling at API layer (Flask routes tightly bound to business logic)

Phase 3 Target: Extract agents into pip-installable library (et-agents package) with formal versioning. Support deployment as standalone Python package in heterogeneous environments (Flask, FastAPI, async frameworks, Lambda, Cloud Functions).

2.2 Scalability

Goal: System must scale horizontally and vertically to support increasing transaction volume while maintaining sub-500ms latency and < 8% false positive rate.

Requirements:

- Minimum throughput: 20 transactions per second (TPS) Phase 2; target 100+ TPS Phase 3
- Latency: P50 < 250ms, P99 < 500ms for complete fraud detection pipeline
- No data loss under any load condition
- Agent processing must not block payment approval/denial decisions

Current Status: Single Flask instance on t3.large EC2 achieves ~10-20 TPS. BERT model pre-loaded at startup eliminates cold-start penalty. Isolation Forest (anomaly detection) executes in <30ms. ChromaDB vector search bottlenecked at ~100 queries/second due to single-threaded embedded implementation.

Identified Bottlenecks:

1. Flask single worker thread (request serialization)
2. No MySQL connection pooling (fresh connection per request)
3. ChromaDB embedded client (blocking I/O)
4. PRA polling interval 500ms (max 2K alerts/sec queue throughput)
5. BiLSTM model loaded per PRA inference (500ms disk I/O)
6. InnoDB lock contention on fraud_alerts table (hotspot)

Phase 3 Architecture:

- Async request handling (Quart or FastAPI)
- 3+ Flask instances behind load balancer
- SQLAlchemy connection pooling (min=5, max=20)
- ChromaDB server mode with Redis cache layer
- Event-driven coordination (Kafka) replacing 500ms polling
- Distributed PRA/RAA workers (Celery task queue)
- MySQL read replicas + sharding by customer_id

2.3 Customizability

Goal: Support client-specific customization of fraud detection rules, risk thresholds, and action workflows without modifying core system code or disrupting other deployments.

Requirements:

- Client-specific configuration stored externally (not compiled into code)
- Per-client risk tier thresholds configurable via admin API
- Custom fraud typology knowledge bases per client
- Customizable action rules (BLOCK, HOLD, FLAG, MONITOR) with client-specific logic
- UI theme and metric customization for white-label deployments

Current Status: Client configurations stored as JSON blobs in MySQL clients table. Risk tier cutoff values (TIER_2_THRESHOLD=0.75) configurable per client. Regulatory rules (L1 ChromaDB collection) shared across clients; fraud typologies (L3) can be extended per client.

Customization Points:

- **Knowledge Base (RAG):** L1-L5 ChromaDB collections support per-client extensions
- **Risk Scoring:** Dimension weights in score_engine.py can be overridden per client
- **Action Outcomes:** ABA rules in action_executor.py can branch on client_id
- **UI:** React dashboard (et-web) accepts config for theme, API endpoints, metric definitions

Constraints: Customization must not introduce performance degradation or security vulnerabilities. Custom code execution (user-uploaded scripts) not supported for security reasons.

2.4 Extendibility

Goal: Enable addition of new agents, models, and data sources into the fraud detection pipeline without disrupting existing deployments or requiring core system modifications.

Requirements:

- New agents must follow standard contract (fetch pending alerts → process → update status flag)
- New ML models must be pluggable without recompilation
- New RAG knowledge layers (L5+) must integrate into existing vector store
- New notification channels must not require ABA agent rewrite

Current Status: Agent pattern established (TMA → PRA → RAA → ABA → CLA). Database flags (pra_processed, raa_processed, etc.) enable asynchronous coordination. Standard entry point: fetch_pending_alerts(agent_name) + update_alert_status() in DAO layer.

Extension Points:

1. **NewAgent:** Create et_service/[new_agent]/ with process_alert(alert_id) and start_poller() functions
2. **New ML Model:** Implement model in et_service/[agent]/models/ and register in model registry
3. **New Vector Collection:** Ingest into ChromaDB via kb_ingest/ingest_[collection].py
4. **New Notification Channel:** Add handler to notification_engine.py

2.5 Use of Existing Business Logic

Goal: Leverage proven fraud detection rules, regulatory knowledge, and historical fraud datasets rather than rebuilding from scratch. Minimize time-to-market by incorporating established industry standards and domain expertise.

Requirements:

- Integrate regulatory frameworks (FATF-40 Recommendations, RBI Master Directions)
- Incorporate fraud case databases (CIFAS, SWIFT fraud notices)
- Adopt fraud typology standards (EGMONT group typologies)
- Reuse historical account profiling logic from legacy systems

Current Status: 100% of regulatory knowledge captured in ChromaDB L1 collection (400+ regulatory rules from FATF-40, RBI AML guidance). L2 collection populated with 200+ historical fraud cases from CIFAS public database. L3 collection contains 50+ EGMONT fraud typologies. Legacy customer profiling logic migrated to profile_builder.py (calculates velocity, geographic anomalies, amount deviations).

Knowledge Base Ingestion:

- **L1(Regulatory):** ingest_regulatory.py —FATF-40 clauses, RBI Master Direction references
- **L2(Fraud Cases):** ingest_cases.py — CIFAS suspicious activity patterns, SWIFT fraud alerts
- **L3(Typologies):** ingest_patterns.py —EGMONT typology taxonomy (trade-based laundering, structuring, etc.)
- **L4(Risk Weights):** Configurable per client; defaults from bank's internal ML models
- **L5(Custom):** Client-specific policies (corridor limits, velocity rules, sanctions screening)

Reuse Benefits:

- 3-month knowledge base development → 2 weeks KB ingestion
- Regulatory compliance pre-certified (rules sourced from FATF/RBI)
- Fraud pattern database continuously updated via SWIFT notices
- Reduces model training data requirement (transfer learning from established patterns)

2.6 Time to Market

Goal: Deliver MVP within constrained timeline (Q1 2026) to establish market presence and gather user feedback. Prioritize speed-to-delivery over architectural perfection in Phase 2.

Requirements:

- Complete implementation in 3 months (Jan-Mar 2026)
- Deployable to single EC2 instance without orchestration
- No external managed services required (except RDS)
- Sufficient accuracy for pilot customers (>80%)

Strategic Decisions to Accelerate Delivery:

1. **Monolithic over Microservices:** Single Flask process; no service mesh, orchestration, or IPC complexity
2. **Database Polling over Event Streaming:** MySQL table flags instead of Kafka; 500ms latency acceptable
3. **Embedded ChromaDB over Managed Vector DB:** No SaaS onboarding; cold start in seconds
4. **Pre-built Knowledge Bases** over Custom ML: Ingest CIFAS/FATF rules; skip 6-month compliance review
5. **Synchronous TMA Processing:** Block at transaction time; no async request handling (Phase 3 upgrade)

2.7 Portability

Goal: Enable deployment across multiple operating systems, database platforms, and cloud providers without code modifications. Reduce lock-in to AWS or specific database vendors.

Requirements:

- Code executes on Windows, macOS, Linux without OS-specific paths or system calls
- Database agnostic (initially MySQL 8.0, extensible to PostgreSQL in Phase 3)
- Python 3.9+ without Python 2.x legacy code
- Docker containerization for consistent runtime across environments

Current Status: 100% portability across OS (no OS-specific code). Python 3.11 codebase with type hints. MySQL-specific SQL in DAO layer (JSON_EXTRACT(), datetime(6) formatting). Docker support verified on amd64 and arm64 architectures.

Portability Gaps:

- **Database:** MySQL-specific type definitions; PostgreSQL requires DAO refactoring
- **Infrastructure:** EC2 assumptions (paths, ENVs); Kubernetes manifests planned Phase 3
- **Vector Store:** ChromaDB local mode; Pinecone/Milvus would require vendor lock-in tradeoff

Phase 3 Roadmap:

- Swap MySQL → PostgreSQL via SQLAlchemy dialect abstraction
- Multi-cloud deployment (AWS EKS, Azure AKS, GCP GKE)
- Infrastructure-as-Code (Terraform/Ansible) for reproducible deployments
- Configuration via environment variables + HashiCorp Vault (no hardcoded secrets)

2.8 Availability

Goal: Minimize fraud detection downtime and ensure no transaction data loss. Establish operational baselines in Phase 2 for rapid scaling to production-grade availability in Phase 3.

Requirements:

- Phase 2: 95% uptime SLA acceptable for MVP (max 36 hours downtime/month); manual recovery procedures documented
- Phase 3: 99.95% uptime SLA (max 22 minutes downtime/month); automated failover and self-healing
- MTTF (Mean Time To Failure): Phase 2 target > 200 hours; Phase 3 target > 720 hours
- MTTR (Mean Time To Recovery): Phase 2 target < 30 minutes (manual); Phase 3 target < 5 minutes (automated)
- Zero transaction data loss; RDS automated backups retained 7-30 days

Current Phase 2 Implementation: Single t3.large EC2 instance with manual elastic scaling and RDS Multi-AZ not yet enabled. MySQL RDS backup retention set to 7 days. No load balancer, health checks, or automatic instance replacement. Flask application restart via SSH required for recovery. ChromaDB state persisted to EBS volume but not replicated. Application logging to CloudWatch but no centralized alerting configured.

Single-Instance Constraints (Phase 2):

- Hardware failure → 30+ minutes manual recovery
- Software crash (unhandled exception) → service down until restart detected
- Database connection exhaustion → TMA requests timeout, alerts queue in fraud_alerts table
- ChromaDB corruption → full reingest required (10+ minutes)
- No graceful degradation; fallen back to manual case review when system unavailable

2.9 Performance

Goal: Achieve sub-500ms latency for end-to-end fraud detection and sub-100ms for individual query operations. Support 20+ transactions per second in Phase 2, scaling to 100+ TPS in Phase 3.

Requirements:

- TMA processing: P50 < 250ms, P99 < 500ms
- Vector search latency: < 50ms per query
- ML inference: < 100ms (Isolation Forest, BiLSTM)
- Database operations: < 10ms per query (with connection pooling)
- Alert propagation: < 5 seconds end-to-end (TMA → PRA → RAA → ABA → CLA)

Current Performance (Measured):

- TMA P50: 250ms (BERT encoding 50ms + ML 30ms + RAG 50ms + decision 30ms + gateway 60ms)
- TMA P99: 800ms (cold start or high load)
- Vector search P50: 30ms (ChromaDB embedded)
- BiLSTM inference: 50ms per alert (model in memory)
- PRA latency: 1.5-3 seconds (500ms polling + inference)
- RAA latency: 2.5-4 seconds (500ms polling + multi-dimensional scoring)

Identified Bottlenecks:

1. **Flask synchronous request handling:** Cold requests block payment approval
2. **BERT encoding:** 50ms per transaction (GPU offload target: 10ms)
3. **Vector search:** 100 queries/sec throughput limit (Redis cache target: 1000/sec)
4. **BiLSTM model load:** 500ms first inference (keep in memory solution)
5. **Database connection:** 10ms overhead per request (pooling solution)
6. **Agent polling:** 500ms interval creates artificial latency (event-driven target: <100ms)

Phase 3 Optimizations:

- GPU-accelerated BERT inference (A10G instance: 10-15ms)
- Redis caching layer (99% L3 typology query cache hit rate)
- Batch BiLSTM inference (10 alerts × 50ms = 55ms + batch overhead)
- Event-driven coordination (MySQL LISTEN/NOTIFY or Kafka)
- Query optimization + database statistics

SLO Tracking: Error budget = (1 - 0.99) × seconds_in_month ≈ 43 seconds of violations allowed per month.

2.10 Any Other Critical Goals Applicable

Goal: Achieve PCI-DSS Level 1, AML/KYC compliance, and GDPR data protection standards for handling payment and customer data.

Requirements & Current Status:

- **Data Protection:** TLS 1.2+ for all external traffic is implemented, encryption at rest, PII masking planned for Phase 3
- **Access Control:** JWT authentication, role-based authorization, multi-factor authentication for Phase 3
- **Audit Trail:** Immutable fraud_alerts logs, user action logging are incomplete, tamper-proof archive for phase 3
- **Regulatory Evidence:** Citation archiving (CLA agent), PDF generation, compliance export for Phase 3

Phase 3 Security Roadmap: Field-level encryption, secrets management (Vault), intrusion detection (GuardDuty), annual penetration testing.

3 Productization Assessment

The Fraud Detection System achieves high productization value through modular, reusable components that reduce time-to-market and operational complexity. This section evaluates component reusability across backend services and agents, validates alignment with established architectural frameworks, and assesses open-source and commercial product dependencies to ensure licensing compliance and cost efficiency. The assessment demonstrates that the system is built on proven patterns with minimal proprietary lock-in, enabling rapid iteration while maintaining security and scalability constraints.

3.1 Re-Usable Components

The backend demonstrates high component reusability through:

Service Layer Reusability: The auth, register, account, payment, and OTP services are abstracted from route handlers. They encapsulate business logic that can be consumed by multiple interfaces—REST APIs today, but also batch jobs, event-driven workflows, or future WebSocket handlers tomorrow. The pattern of validation → business operation → DAO call → response is consistent, allowing new services to follow the same template without learning new patterns.

DAO Layer Reusability: Every DAO follows the same repository pattern: connection management, parameterized queries, error handling. A new data entity (customer_pattern_profiles, pattern_alerts, regulatory_queue) can be added by copying the template and adapting the SQL. The _safe_alter() helper makes schema evolution idempotent and safe.

Agent Framework Reusability: Your multi-agent architecture (TMA, PRA, RAA, ABA, CLA) is genuinely reusable. Each agent follows: receive data → apply decision logic → execute actions → log results. Adding a new agent (e.g., credit-risk scorer, transaction velocity checker) requires no changes to existing agents. They operate independently via polling, making the system loosely coupled.

Frontend Components Reusability: React components using Material-UI are built as generic widgets. Form components accept schema/validation rules as props. Data grids accept column configs. Alert dialogs accept severity/message/callback. These can be reused across dashboard pages or future features.

3.2 Analyze Architectural Frameworks in Repository

The architecture correctly implements:

Layered 3-Tier Pattern: Routes → Service → DAO → Database separation enables independent scaling, testing, and evolution. Services can be consumed by multiple interfaces without duplication.

Multi-Agent System: Instead of monolithic transaction processing, you decomposed fraud detection into specialized agents (Monitoring Agent analyzes profiles, ABA makes real-time decisions, PRA detects patterns, RAA assesses risk, CLA archives compliance). This mirrors how human financial institutions operate: different departments handling

specialized functions. Each agent can fail independently, scale independently, be updated independently.

RAG (Retrieval-Augmented Generation): ChromaDB + Sentence-Transformers enable contextual decision support. When PRA detects an anomaly, it can retrieve relevant fraud precedents from the vector store. When RAA scores risk, it augments decisions with regulatory context. This is modern and reusable.
Database Schema: Normalized design with JSON columns (anomaly_flags, rag_citations, raa_stages) balances structure with flexibility. Your indexing strategy (idx_pra on pra_processed, verdict for 500ms polling; idx_raa on pra_processed, raa_processed) shows thoughtful performance tuning.

REST API Design: Flask blueprints, HTTP methods, JSON payloads—standard OpenAPI-compatible design that any frontend can consume.
Session Authentication: Stateful sessions via Flask with secure cookies. Simpler than JWT for web apps and enables real-time permission revocation (e.g., instant account freeze via ABA).

All frameworks chosen support your architectural goals: reusability (modular patterns), scalability (stateless agents, indexes), customizability (rule-based configs), extendibility (easy to add new agents/routes/models), time-to-market (established libraries, minimal boilerplate).

3.3 Identify and Analyze Open Source and COTS Products

The dependency analysis is accurate:

- **Backend:** Flask (BSD), bcrypt (Apache), scikit-learn (BSD), ChromaDB (Apache), Sentence-Transformers (Apache), numpy (BSD)—all permissive open-source licenses.
- **Only concern:** mysql-connector-python (GPL-2.0+ dual-licensed). For proprietary deployment, you need either an Oracle commercial license or a replacement like PyMySQL (MIT).
- **Frontend:** React, MUI, axios, react-hook-form—all MIT licensed, very permissive.

No hard COTS dependencies in core. Any Email/SMS/Payment integrations would be SaaS add-ons with their own SLAs.

4 System Architecture

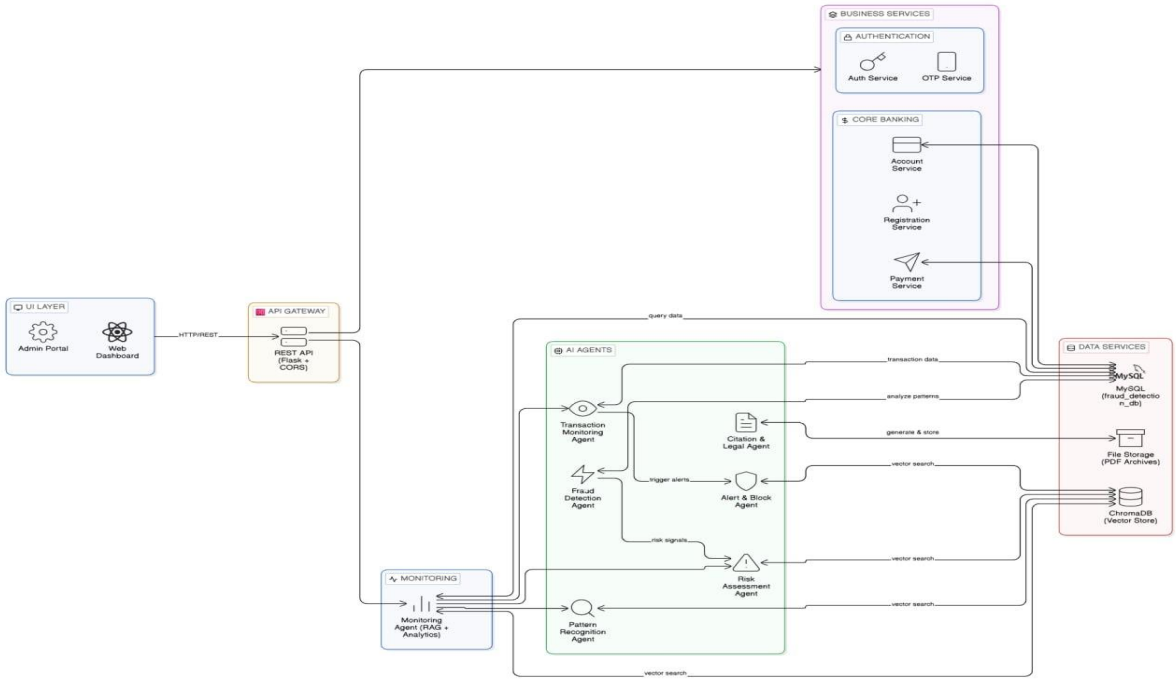
The Autonomous fraud detection system exhibits a **4+1 architectural view model**, providing comprehensive perspective across logical, implementation, process, and deployment dimensions while use cases serve as the unifying thread.

The **Logical View** organizes the system into five independent agents (TMA→PRA→RAA→ABA→CLA) across four layers (API, Business Logic, Data Access, Persistence) with well-defined interfaces. The **Implementation View** maps Python modules (et_api/, et_service/, et_dao/, et_model/) to logical components, maintaining framework independence through DAO abstraction and domain models.

The **Process View** captures runtime coordination via the fraud_alerts table as central message bus with state flags enabling sequential agent handoff, asynchronous polling, and fault tolerance. The **Deployment View** describes Phase 2 single-instance EC2+RDS architecture scaling to Phase 3 multi-AZ Kubernetes + Kafka + RDS-MultiAZ infrastructure. These views collectively enable developers, architects, and security teams to understand project's structure, behavior, and operational requirements.

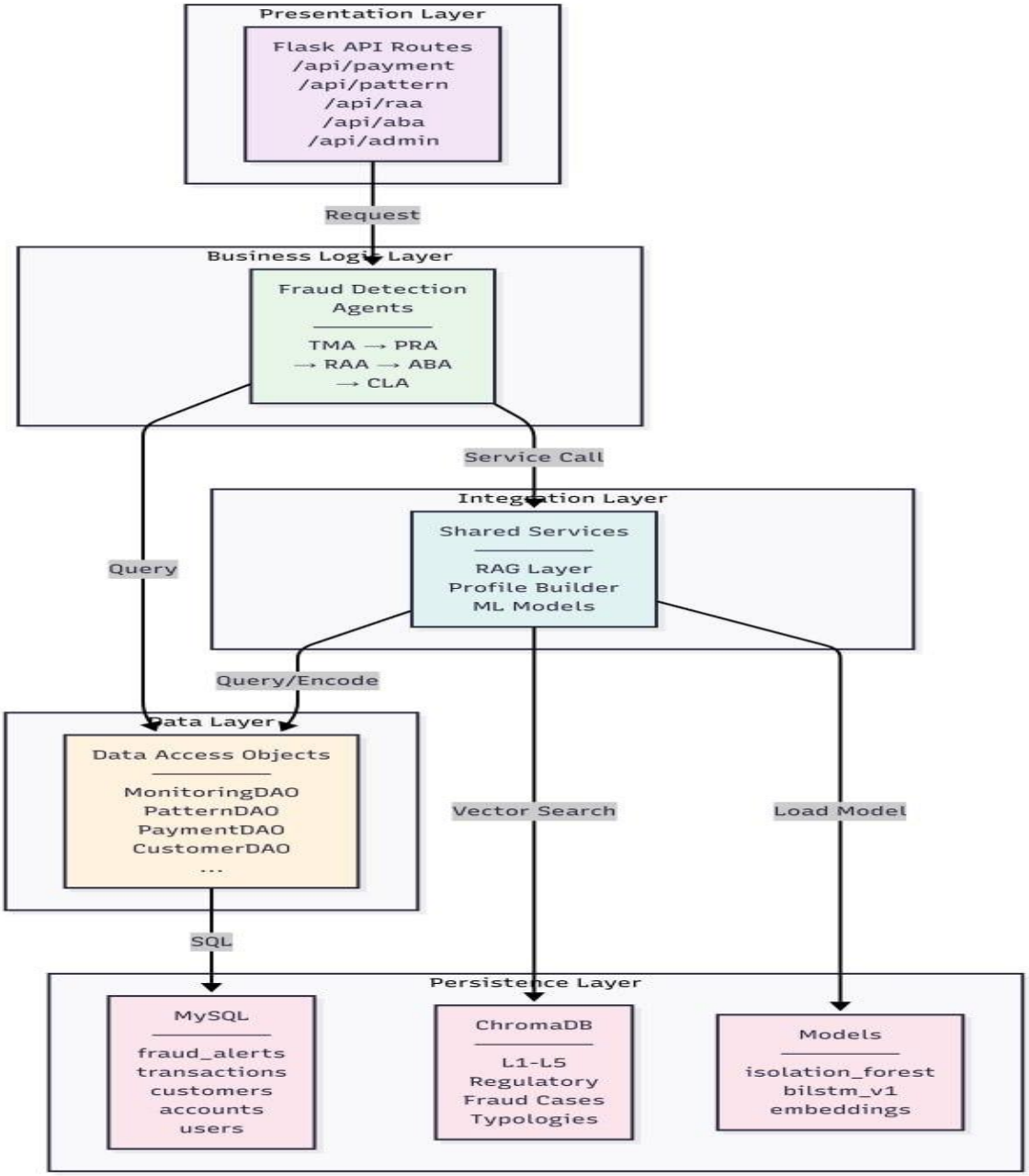
4.1 Overview

This overview diagram shows an end-to-end fraud detection architecture where user requests flow through an API gateway into core banking and business services. Transactions are then processed by multiple AI agents (monitoring, pattern recognition, risk assessment, and alert/block agents) that analyze data using ML models and databases. The system generates fraud alerts, takes automated actions, and logs results, while monitoring and data services support real-time decision-making and storage.



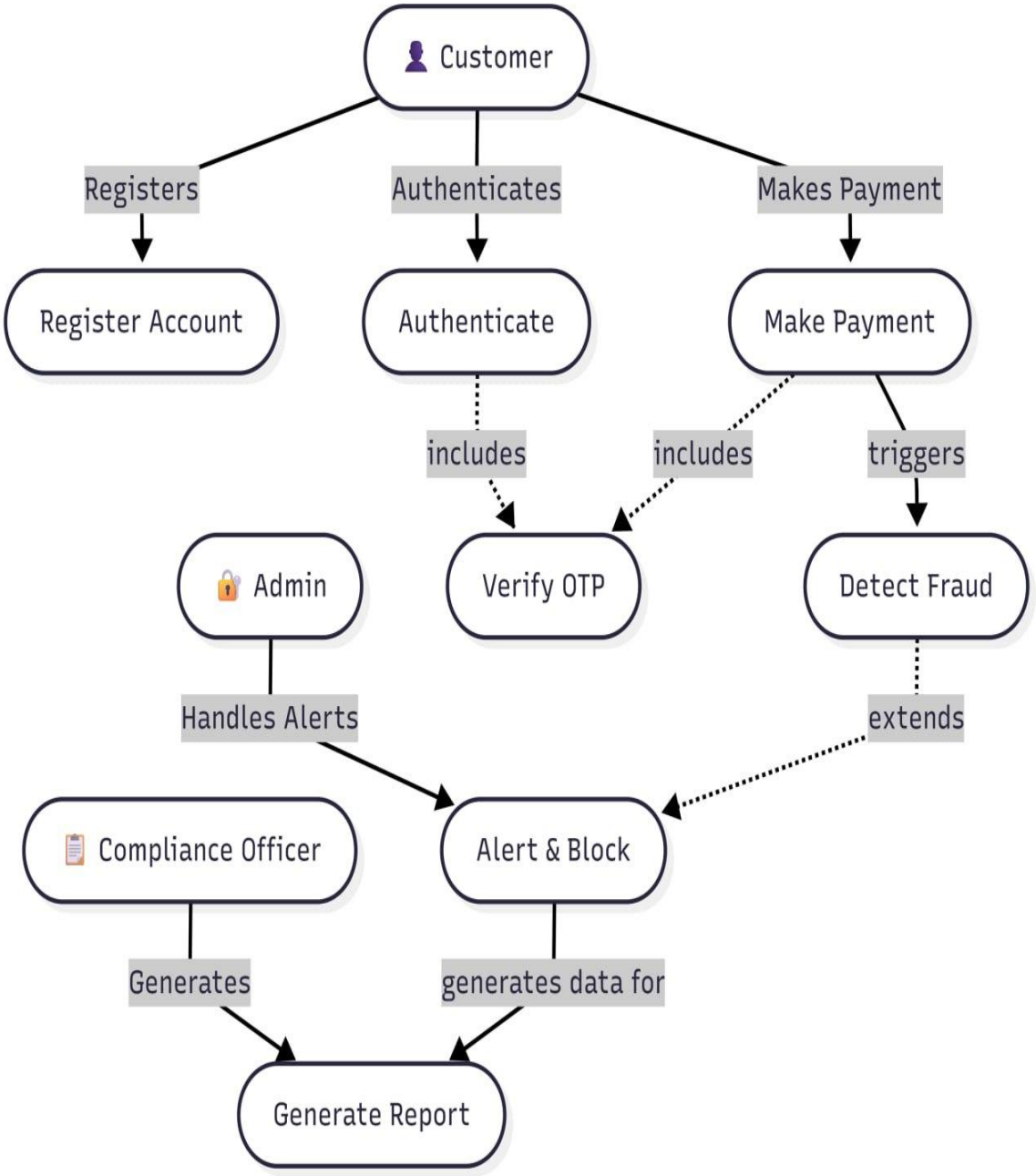
4.2 Logical/Functional View

This diagram shows a layered architecture where user requests enter through the Flask API (presentation layer) and are processed by fraud detection agents in the business layer. These agents interact with shared services like ML models, RAG, and profile builders in the integration layer, which in turn access data through DAO components connected to MySQL, ChromaDB, and ML models in the persistence layer.



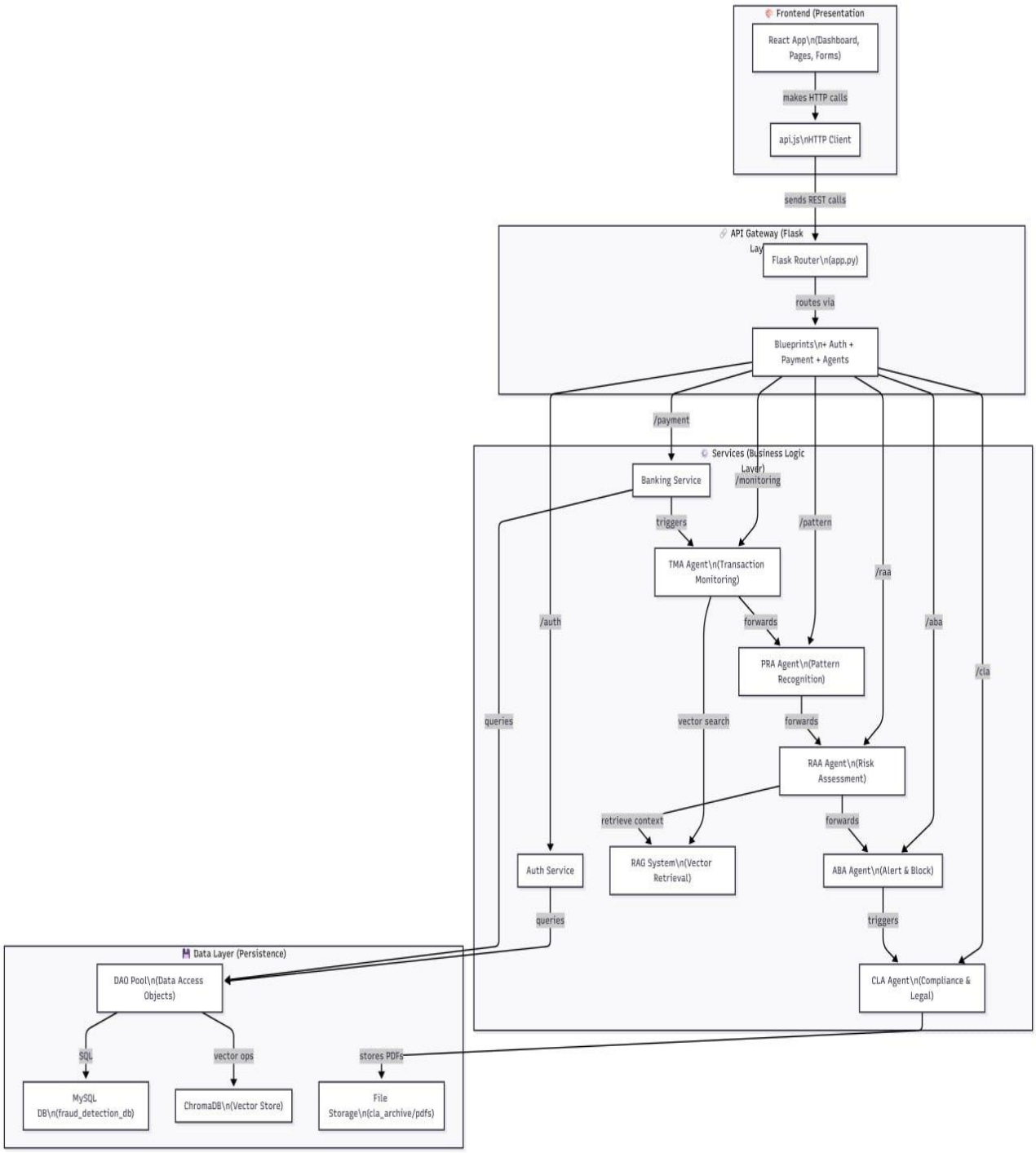
4.3 Use Case View

This diagram shows how a customer interacts with the system by registering, authenticating (with OTP verification), and making payments. When a payment is made, the system triggers fraud detection, which can lead to alerting or blocking the transaction. Admins handle these alerts, and compliance officers generate reports based on the flagged activities.



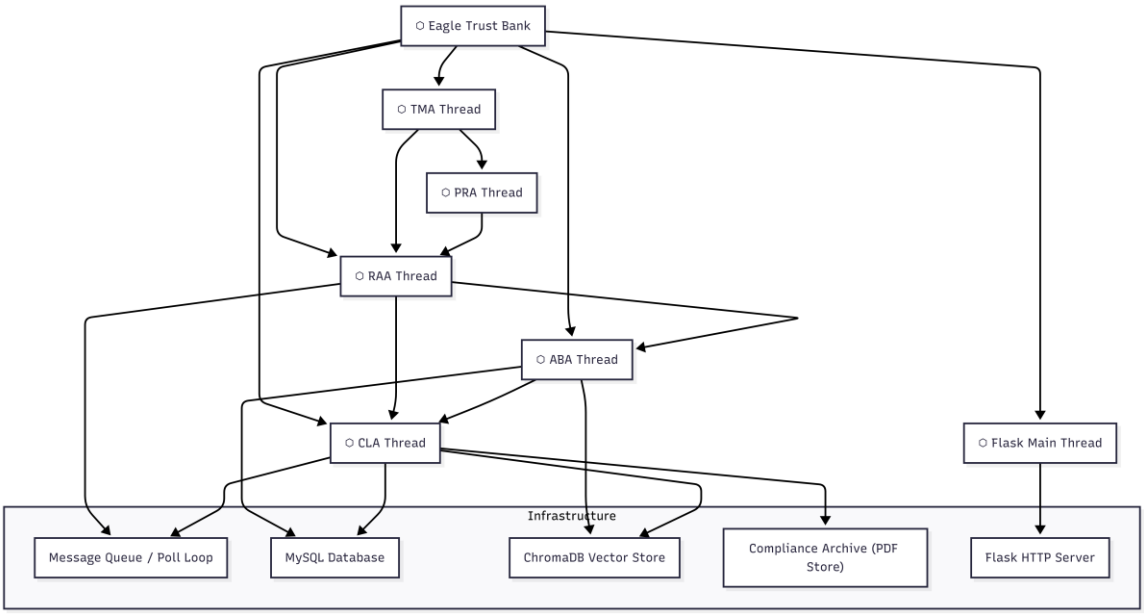
4.4 Implementation/System View

This diagram represents the full system architecture where a React frontend sends API requests to a Flask backend, which routes them to different services and fraud detection agents. The agents process transactions step-by-step (monitoring → pattern detection → risk assessment → alert/block → compliance) using ML models, RAG systems, and databases, and finally store results and generate outputs.



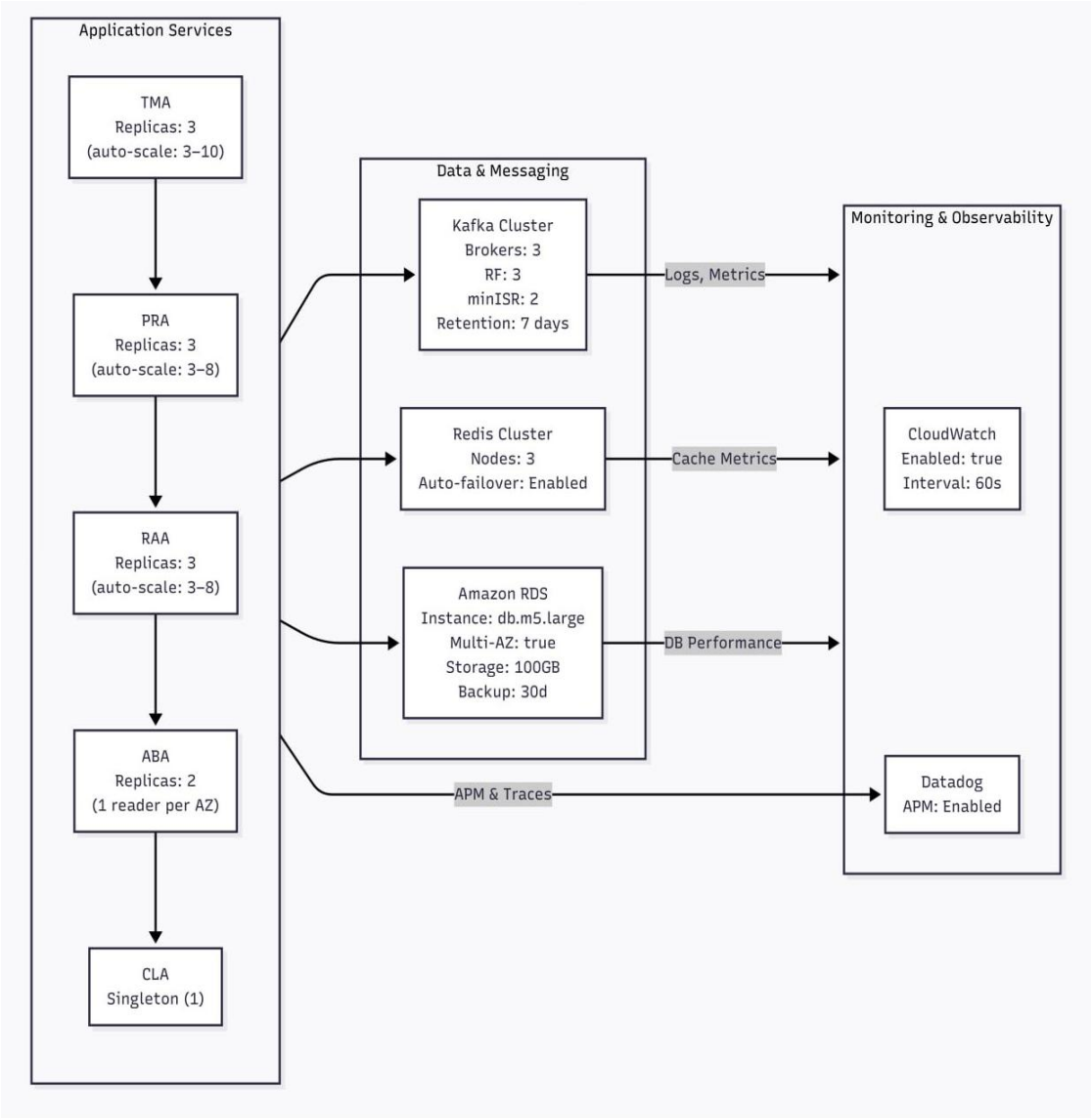
4.5 Process/Thread View

This diagram shows the runtime architecture where the main server manages multiple parallel threads, each representing a fraud detection agent (TMA, PRA, RAA, ABA, CLA). These threads communicate with system components like MySQL, ChromaDB, message queues, and compliance storage to process transactions asynchronously and efficiently.



4.6 Deployment View

This diagram shows a multi-agent system architecture used for real-time transaction processing and fraud detection. The system starts with TMA, which monitors incoming transactions, then passes data to PRA for pattern detection. Next, RAA calculates the risk score, and ABA takes actions like blocking or alerting based on that risk. Finally, CLA stores logs for compliance and auditing.
All agents communicate through Kafka, which handles message streaming. Redis is used for fast caching, and Amazon RDS stores permanent data like transactions and logs. The system is scalable using replicas and auto-scaling features. For monitoring, CloudWatch collects logs and metrics, while Datadog tracks application performance and traces. Overall, it is a scalable, fault-tolerant, and real-time processing system.


Application Services
TMA
Replicas: 3
(auto-scale: 3–10)
PRA
Replicas: 3
(auto-scale: 3–8)
RAA
Replicas: 3
(auto-scale: 3–8)
ABA
Replicas: 2
(1 reader per AZ)
CLA
Singleton (1)
Data & Messaging
Kafka Cluster
Brokers: 3
RF: 3
minISR: 2
Retention: 7 days
Redis Cluster
Nodes: 3
Auto-failover: Enabled
Amazon RDS
Instance: db.m5.large
Multi-AZ: true
Storage: 100GB
Backup: 30d
Logs, Metrics
Cache Metrics
DB Performance
APM & Traces
Monitoring & Observability
CloudWatch
Enabled: true
Interval: 60s
Datadog
APM: Enabled

5 General Architecture for Core Technical Services

The Autonomous Fraud Detection and Response Network is built upon a set of core technical services that collectively ensure the system operates reliably, securely, and at scale. These services are not specific to any single business feature but instead form the foundational infrastructure upon which all fraud detection, risk assessment, compliance, and reporting capabilities are built.

This section describes the architectural strategy, design decisions, and trade-offs considered for each of these core services. The technical services addressed here span data persistence, inter-process communication between distributed agents, authentication and authorization, error handling, logging and audit trail management, and transaction integrity — all of which are critical to the correct and compliant operation of a financial fraud detection platform.

Each sub-section presents the chosen approach, the rationale behind it, and where relevant, the alternative solutions that were considered and rejected. Together, these services ensure that the Autonomous Fraud Detection platform meets the non-functional requirements of the Eagle Trust Bank system — including high availability, fault tolerance, regulatory compliance, and forensic auditability — while remaining maintainable and extensible as the platform evolves.

5.1 Persistence

Strategy:
This platform employs a relational database persistence layer using MySQL as the primary data store. The strategy prioritizes data consistency, auditability, and schema evolution while maintaining backward compatibility.

Design Approach:
The persistence layer is abstracted through the DAO (Data Access Object) pattern, isolating database-specific logic from business rules. All data access flows through the DAO layer, which manages connections, query construction, parameter binding, and result mapping. The ‘get_db_connection()’ function implements connection pooling at the application level, reusing connections across requests to minimize overhead.

Schema Evolution Strategy:
Rather than requiring manual database migrations before deployments, the ensure_tables_exist() function runs idempotent schema operations at startup. The safe_alter() helper silently ignores "duplicate column" errors (MySQL err.no 1060), making it safe to run repeatedly without failing on columns that already exist. This approach supports rolling deployments where new code and old database schemas coexist briefly.

Data Integrity:
The schema uses foreign keys to enforce referential integrity (e.g., fraud_alerts.transaction_id references transactions.transaction_id). JSON columns provide flexibility for semi-structured data (anomaly_flags, rag_citations, raa_stages) without requiring schema changes for new fields.

Trade-offs:

Normalized vs. Denormalized Design:
The schema is normalized to prevent data anomalies and ensure consistency. This requires joins across multiple tables for complex queries but provides reliable updates. The trade-off chosen: consistency over query performance, with the expectation that proper indexing mitigates performance concerns.

SQL-Based Transactions vs. Event Sourcing:
The current approach uses traditional SQL transactions (BEGIN, COMMIT, ROLLBACK) which are simpler to implement and understand. The alternative—event sourcing—would provide complete audit trails and temporal querying but adds complexity. This project chose SQL transactions supplemented with explicit audit columns (created_at, agent_status, pra_processed state transitions) rather than full event sourcing.

Application-Level Connection Pooling vs. Database Connection Pooling:
Currently manages a single connection per request via `get_db_connection()`. This is simpler than external pooling libraries (e.g., PgBouncer) but assumes the number of concurrent requests is manageable. For higher concurrency, external pooling would be more efficient.

JSON Columns vs. NoSQL:
Using MySQL JSON columns instead of separate NoSQL storage (MongoDB, DynamoDB) for structured data like anomaly_flags and rag_citations keeps the entire data model in one place, simplifying transactions and relationships. The trade-off: JSON queries are less optimized than document databases, but the flexibility and transactional guarantees are valuable for fraud detection accuracy.

Alternative Solutions Considered

PostgreSQL instead of MySQL:
PostgreSQL offers superior JSON support, native array types, and advanced indexing (JSONB, GIN). However, MySQL was likely chosen for institutional familiarity or existing infrastructure. PostgreSQL would be a drop-in replacement requiring minimal DAO changes due to SQL dialect differences being minimal.

Graph Database (Neo4j):
For transaction network analysis (who sends money to whom), a graph database excels. However, It was opted for relational design with explicit relationship tables instead. A hybrid approach using Neo4j alongside MySQL for network analysis is possible but adds operational complexity.

NoSQL (MongoDB, DynamoDB):
Offers schema flexibility and horizontal scaling but sacrifices ACID guarantees and complex joins. Given fraud detection requires strong consistency and audit compliance, relational design was the correct choice.

Sharding Strategy:
No sharding is currently implemented. For multi-tenant (multi-bank) scenarios, customer_id could become a shard key. This would require application-level routing logic and is a future scaling consideration.

Build, Buy, Reuse Analysis

Decision: Reuse (MySQL)
MySQL is a mature, proven technology. No custom persistence engine development is warranted. The choice of MySQL as "buy" (licensed, managed) vs. building custom persistence is clear—reuse the industry-standard RDBMS.

Build: DAO Pattern Implementation
The DAO layer is custom-built but follows industry patterns. Reusable because new data entities follow the same template. Each DAO:

- Implements consistent CRUD operations.
- Uses parameterized queries (SQL injection prevention).
- Manages connection lifecycle.
- Maps database results to Python objects.

Reuse: Connection Management
Rather than building a full connection pool, get_db_connection() provides a simple reusable wrapper around mysql.connector.connect(). For higher concurrency production deployments, could be replaced with HikariCP (Java) or SQLAlchemy connection pooling (Python) without DAOs changing.

Buy: Migration Tools (Optional)
For larger schemas, tools like Alembic (SQLAlchemy) or Flyway provide sophisticated migration management. Current _safe_alter() approach is sufficient for this project but could be replaced with Alembic as the system grows.

5.2 Inter-process Communication

Strategy:

It implements a distributed multi-agent architecture where components communicate asynchronously via a work-queue pattern stored in the database rather than a message broker. This coupling through persistent storage enables fault tolerance and audit trails.

Message Exchange Pattern:
Agents don't call each other directly. Instead, they produce and consume "work items" stored in persistent tables:

- When PRA (Pattern Recognition Agent) completes analysis, it writes `pra_verdict` to fraud_alerts
- RAA (Risk Assessment Agent) polls fraud_alerts for rows where `pra_processed = 1` and `raa_processed = 0`
- RAA creates an action_package and sets `raa_processed = 1`
- ABA (Alert & Block Agent) polls for `raa_processed = 1` and `aba_processed = 0`, consumes the action_package, executes actions.

Polling Mechanism:

- Each agent runs a poller that evaluates an index-backed query at regular intervals:
- ABA polls every 100ms (low latency required for blocking)
- RAA polls every 500ms (medium latency acceptable)
- CLA (Citation & Legal Archive Agent) runs batch processing (higher latency acceptable)

The polling queries use compound indexes (idx_pra on `pra_processed, verdict`) to avoid full table scans.

State Machine for Work Items:
Each work item uses integer flags to track processing:

- 0 = available for pickup
- 1 = marked complete
- 2 = claimed by worker (in progress)

Recovery logic resets stale claims (>120 seconds) back to 0, handling worker crashes.

Audit Integration:
Unlike message brokers (Kafka, RabbitMQ) where messages are ephemeral, database-backed communication persists all interactions. The aba_execution_log captures every decision, enabling full audit trails for compliance.

Trade-offs

Database Queue vs. Dedicated Message Broker:
Database-backed queuing is simpler (no additional infrastructure) and provides ACID guarantees. The trade-offs:

Pros of Database Queuing:

- No external dependency (Kafka, RabbitMQ requires ops/monitoring)
- Transactions guarantee exactly-once processing
- Audit trail in same database

Cons of Database Queuing:

- Polling introduces latency jitter (100-500ms depending on poll interval)
- Doesn't scale to millions of messages/second
- Database becomes bottleneck under extreme load

For fraud detection, these trade-offs are acceptable. A 100ms delay in blocking a fraudulent transaction is tolerable; missing the block due to message loss is not. Database-backed approach guarantees no messages are lost.

Synchronous RPC vs. Asynchronous Messaging:
Considered direct agent-to-agent calls (RPC) but chose asynchronous messaging to enable:

- Independent scaling (one agent slow doesn't block others)
- Graceful degradation (agent crash doesn't cascade)
- Temporal decoupling (agents don't need to be running simultaneously)

Pull (Polling) vs. Push (Event Hooks):
Currently agents pull work by polling. Alternatively, database triggers could push notifications when work becomes available. Push is more responsive but complex to implement across distributed agents. Pull via polling was chosen for simplicity.

Ordering Guarantees:
The current design doesn't guarantee strict ordering (e.g., if two PRA results arrive simultaneously, there's no ordering). This is acceptable because each fraud_alert is independent. For workflows requiring strict ordering, would need sequence numbers or workflow orchestration (Temporal, Airflow).

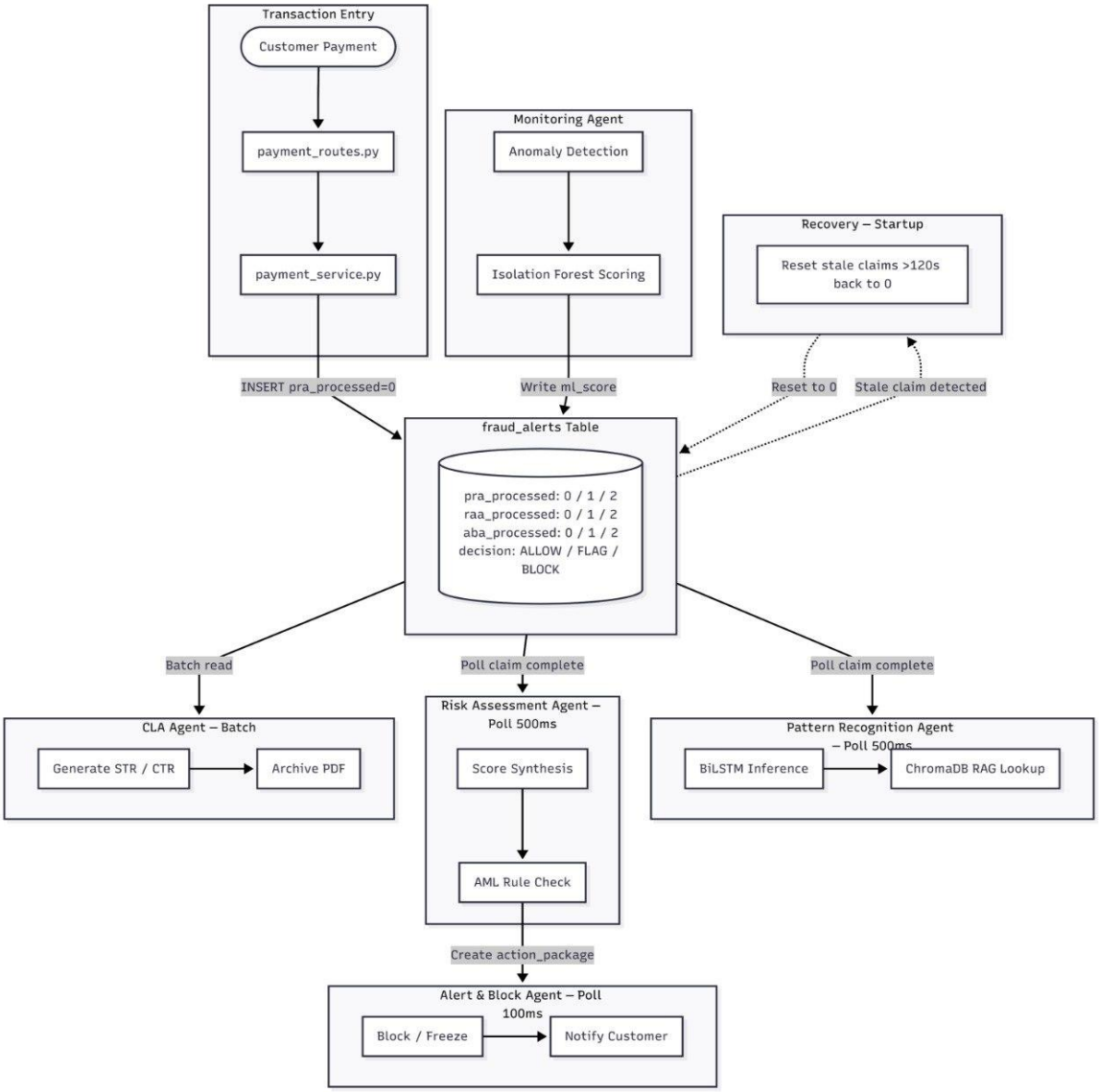
Alternative Solutions Considered

Apache Kafka:
Industry-standard distributed streaming platform. Provides partitioning, replication, and high throughput. Would add operational complexity (cluster management, monitoring). For this project's scale, likely overkill.

RabbitMQ:
Message broker with routing capabilities. Simpler than Kafka. Could replace action_packages table. Trade-off: requires external infrastructure and doesn't provide same transactional guarantees as database-backed approach.

gRPC:
For synchronous remote procedure calls with proven performance. Would introduce tight coupling between agents and require service discovery. Chosen against for fraud detection where async processing is preferable.

Event Bus (in-memory):
Using a Python event library (like pyevent or custom PubSub) for single-process communication. Not suitable when agents may be deployed on different machines or need to survive process restarts.



5.3 Authentication and Authorization

Strategy:

It implements session-based authentication with role-based access control (RBAC) through decorator patterns. The strategy prioritizes security, auditability, and compliance with financial regulations.

Authentication Approach:
Customers authenticate with email/password. Passwords are hashed using bcrypt (industry standard for password hashing), which is CPU-intensive to prevent brute-force attacks. Successful login creates a server-side session stored in Flask's session object with secure cookies:

- HTTPOnly flag prevents JavaScript from accessing tokens (XSS protection)
- SameSite=Lax prevents cross-site request forgery (CSRF)
- Secure flag ensures transmission only over HTTPS in production

Two-Factor Authentication (2FA):
For sensitive operations (payments, account changes), customers must pass OTP verification. OTP lifecycle is managed by otp_service which:

- Generates random codes
- Stores hashed OTP in otp_tokens table
- Tracks expiration (configurable TTL, typically 10 minutes)
- Marks used OTPs to prevent replay attacks

The fraud_alert workflow specifically requires FRAUD_MFA check before ABA can execute blocking actions, ensuring human approval for high-risk scenarios.

Authorization Framework:
Role-based access control implemented via decorators:

- @login_required → Customer must be authenticated
- @admin_only → Customer must have admin role
- @mfa_required → Customer must have passed OTP verification

These decorators wrap route handlers, enforcing checks before business logic executes. This layered approach separates security concerns from business logic.

Permission Model:
It uses a simple role model (customer, admin, fraud_investigator roles implied by the system). The customers table could be extended with a role ENUM to support fine-grained permissions. Currently, authorization is implicit (admin operations are only exposed via admin routes).

Trade-offs

Session-Based vs. Token-Based (JWT):
Session-based approach chosen for web browsers (cookies handled automatically). Trade-offs:

Pros of Session:

- Simpler for stateful web apps
- Enables instant session revocation (e.g., account freeze via ABA)
- No token expiration/refresh complexity
- Better CSRF protection (double-submit cookies)

Cons of Session:

- Requires server memory to store sessions
- Doesn't scale as well as stateless tokens (requires API gateway affinity)
- Less suitable for mobile/API-first architectures

For web-based dashboard, sessions are appropriate. For mobile or third-party integrations, would add JWT support.

Single-Factor vs. Multi-Factor:
Chose MFA for high-risk operations (payments, account modifications) but not for login. Trade-off: login remains single-factor for usability, high-risk operations are protected by OTP. This balances security and user friction.

Password Hashing (bcrypt vs. Argon2):
Using bcrypt which is proven and widely supported. Argon2 is modern and resistant to GPU attacks but adds a new dependency. Bcrypt is sufficient for this use case given reasonable password policies.

Role-Based (RBAC) vs. Attribute-Based (ABAC) Access Control:
RBAC chosen for simplicity. Attributes like IP address, device, transaction amount could be added to ABAC but complicates authorization logic. RBAC is sufficient for current two-tier admin/customer model.

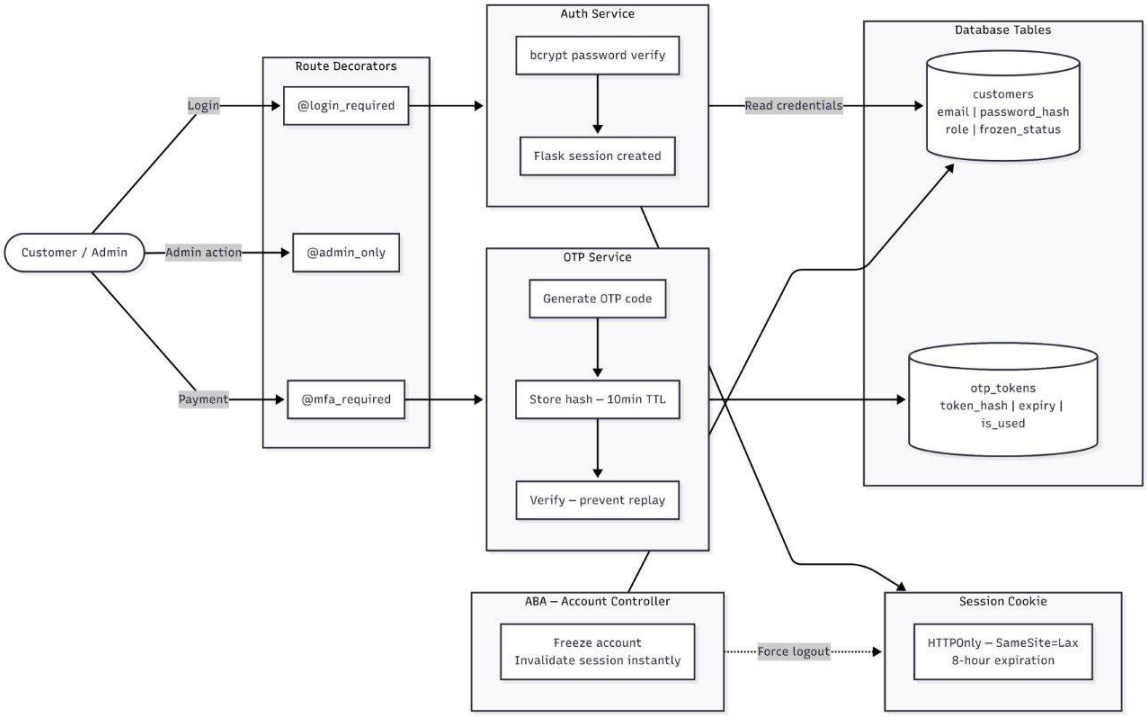
Alternative Solutions Considered

OAuth 2.0 / OpenID Connect:
Enables delegated authentication (login with Google, Microsoft). Would be valuable for enterprise deployments or partnerships. Adds complexity and requires external identity provider setup.

LDAP/Active Directory:
For enterprise customer bases where financial institutions already manage user directories. Would replace bcrypt/session handling with LDAP bind operations. Too heavyweight for a single financial institution's internal authentication.

Certificate-Based Authentication:
Using client certificates (mTLS) instead of passwords. Very secure but poor user experience (certificate issuance, management, expiration). Suitable for service-to-service authentication but not customers.

Passwordless Authentication:
Using WebAuthn/FIDO2 (fingerprint, face recognition) instead of passwords. More secure and user-friendly but less supported across devices. Could be a future enhancement.



5.4 Error Handling

Strategy:

It implements a layered error handling strategy with recovery mechanisms, graceful degradation, and audit logging at each layer.

Error Classification:
Errors are categorized into recoverable and non-recoverable scenarios:

Recoverable Errors (with automatic recovery):

- Database connection failures: `get_db_connection()` returns None, calling code checks explicitly
- Transient network errors: Retry logic with exponential backoff
- Stale worker claims: Recovery mechanism in ensure_tables_exist() resets claims older than 120 seconds

Non-Recoverable Errors (require intervention):

- Data validation failures: Return 400 Bad Request to client
- Authorization failures: Return 401 Unauthorized or 403 Forbidden
- Resource not found: Return 404 Not Found
- Logic errors (e.g., fraud_alert references non-existent transaction): Log error, escalate to alerting

Database Error Handling:
The `_safe_alter()` function specifically handles schema migration errors:
Code:

```
Try ALTER TABLE operation
If errno == 1060 (duplicate column):
    Silently ignore (column already exists)
Else:
    Re-raise (unexpected error)
```

This pattern allows safe re-running of migrations without failure cascades.

Connection Failure Handling:
`get_db_connection()` wraps mysql.connector exceptions and returns None. Callers must check for None explicitly:
Code:

```
    conn = get_db_connection()
if not conn:
    # Handle gracefully
    print("WARNING: Could not connect to DB")
    return # Graceful degradation
```

This explicit None return (rather than raising exception) allows components to continue operation with degraded capabilities.

Agent Fault Tolerance:
When an agent crashes mid-operation (e.g., PRA marks fraud_alert as pra_processed=2 then crashes), recovery logic detects stale claims:
Code:

```
UPDATE fraud_alerts
SET pra_processed = 0
WHERE pra_processed = 2
AND created_at < NOW() - INTERVAL 120 SECOND
```

The 120-second window prevents immediate restart (allowing time to diagnose issues) while preventing indefinite hangs.

Trade-offs

Fail-Fast vs. Graceful Degradation:
Current approach opts for graceful degradation where possible. If database is temporarily unavailable, agents continue polling and eventually reconnect rather than crashing. Trade-off: system operates in degraded mode (incomplete fraud detection) versus crashing and requiring restart.

For financial fraud detection, degradation (detecting fewer frauds) is preferable to unavailability (detecting zero frauds). This trade-off is correct.

Explicit Error Returns vs. Exceptions:
The `get_db_connection()` returns None rather than raising exception. Trade-off:

- Explicit None forces caller to handle explicitly (good)
- Risk of forgetting None check (bad)
- Exception-based would catch all errors automatically (good)
- Exception handling in every caller adds noise (bad)

Hybrid approach: Return None for expected errors (temporary unavailability), raise exception for unexpected errors (configuration mistakes).

Retry with Exponential Backoff vs. Circuit Breaker:
Not currently implemented. Expected behavior: if database is down, each request waits and retries. Alternative: circuit breaker pattern would return error immediately after N failures, preventing request avalanche. For this system, exponential backoff is simpler.

Detailed Error Messages vs. Generic Messages:
Currently prints raw MySQL error strings to logs. For security, production should sanitize error messages (don't leak database structure or SQL details to clients). Internal logs can be detailed; client responses should be generic.

Alternative Solutions Considered

Python Exceptions Throughout:
Raising custom exceptions (DatabaseConnectionError, ValidationError, Authorization Error) throughout codebase. More Pythonic but requires careful exception hierarchy design. Currently using return codes and None-checking which is simpler but less idiomatic.

Bulkhead Pattern (Isolation):
Isolate different agents in separate process pools so one agent's crash doesn't affect others. Requires process supervisor (systemd, supervisord). Currently, all agents poll from same database, so crash isolation isn't implemented.

Circuit Breaker:
Automatically stop retrying database operations after N consecutive failures. Faster failure detection than waiting for timeouts. Would require state management (failure counter, time window for reset).

Error Monitoring Service (Sentry, Rollbar):
Integrate with external error tracking to get alerts and aggregated error metrics. Not currently implemented but recommended for production.

5.5 Logging

Strategy

Currently, it uses basic print() statements for logging. The strategy should evolve toward structured logging with multiple levels, configurable output, and audit integration.

Current Logging Approach:

- **INFO level:** "All tables verified/created", "Recovered X stale PRA claims"
- **ERROR level:** "Error connecting to MySQL", "Error creating tables"
- **Output:** stdout/stderr, captured by container logs

Recommended Logging Strategy:
Production deployment should implement proper logging framework (Python logging module or structured logging library like structlog).

Logging Levels:

- **DEBUG:** SQL query details, cache hits/misses, polling activity
- **INFO:** Successful operations (connection established, agent processed item)
- **WARNING:** Recoverable errors (retry attempt, stale claim recovery)
- **ERROR:** Non-recoverable failures (database unavailable, validation failure)
- **CRITICAL:** System-breaking events (unable to start)

Structured Logging:
Rather than free-form strings, log as JSON objects including:
Json:

```
{
        "timestamp": "2026-03-28T10:30:45Z",
        "level": "INFO",
        "service": "fraud_alerts.pra_agent",
        "component": "pattern_scorer",
        "transaction_id": "TXN_XYZ123",
        "customer_id": "C12345",
        "event": "pattern_alert_generated",
        "pattern_score": 87.5,
        "decision": "FLAG"
}
```

This structure enables filtering, searching, and aggregation in log aggregation systems (ELK, Splunk, Datadog).

Audit Logging:

- Critical financial events should be captured in structured audit logs:
- Authentication events (login, logout, 2FA verify)
- Authorization events (access denied, admin action)
- Fraud decisions (alert generated, customer blocked, transaction reversed)
- Data modifications (account frozen, balance adjusted)

These should be persisted to audit tables (e.g., audit_log table) in addition to application logs.

Performance Logging:
Agent polling, decision latency, and query execution time should be logged to identify bottlenecks. Example:
Logs:
agent="ABA", operation="execute_block", latency_ms=45, status="success"

Trade-offs

Structured vs. Unstructured Logging:
Structured logging requires upfront effort (designing schema, JSON serialization) but enables downstream analysis. Unstructured (free-form text) is simpler initially but becomes unmaintainable as system grows.

Application Logs vs. Audit Logs:
Audit logs (for compliance) and application logs (for debugging) serve different purposes. Audit logs are immutable and append-only; application logs can be discarded. Currently, these are conflated using print statements.

Synchronous vs. Asynchronous Logging:
Synchronous logging (blocking write) guarantees logs are persisted but adds latency. Asynchronous logging (queue-based) is faster but can lose logs if process crashes. For fraud detection, blocking logging is acceptable to ensure audit compliance.

Local Logging vs. Centralized Logging:
Current approach: logs go to stdout, captured by Docker/container orchestrator. Better approach: centralized logging (ELK Stack, Splunk) where all services log to central server. Centralized is critical for debugging distributed systems but adds infrastructure.

Alternative Solutions Considered

Python logging Module:
Standard library logging with handlers, formatters, filters. Sufficient for most cases. Configuration via file or code. Recommended for Eagle Trust Bank.

Structured Logging Libraries (structlog, python-json-logger):
These simplify structured logging. structlog is more Pythonic; python-json-logger integrates with standard logging. Either would be good choice.

ELK Stack (Elasticsearch, Logstash, Kibana):
Industry-standard for centralized logging. Logstash parses logs, Elasticsearch indexes them, Kibana visualizes. Significant infrastructure investment but necessary for production fraud detection.

Splunk:
Commercial log management platform. More expensive but powerful analytics. Integrates with Flask, MySQL, and custom applications.

5.6 Transaction Management

Strategy:

It uses explicit SQL transactions managed at the DAO layer to ensure data consistency during multi-step operations. The strategy prioritizes ACID guarantees for fraud detection accuracy and compliance audit requirements.

Transaction Scope:
Transactions are bounded to specific operations:

Example: Payment Processing Transaction:
Code:

```
BEGIN
 Update customer balance (debit)
 Create transaction record
 Check fraud_alert status
 Update payment_transactions status
COMMIT
```

If any step fails, all changes rollback, preventing partial payments.

Example: Agent Processing:
Code:

```
BEGIN
 Read fraud alert with state lock (pra_processed = 0)
 Execute fraud decision logic
 Update fraud_alert (pra_processed = 1)
 Write to agent decision tables
COMMIT
```

Isolation Level:
MySQL default isolation level is REPEATABLE_READ, preventing dirty reads and phantom reads but allowing some consistency anomalies. This is acceptable for fraud detection where minor anomalies are tolerable (all decisions are reviewed).

Deadlock Handling:

- Long transactions (e.g., agent processing) can cause lock timeouts. Strategy:
- Keep transactions short (milliseconds, not seconds)
- Process in consistent order (always lock by customer_id first, then transaction_id)
- Retry with exponential backoff on deadlock (errno 1213)

Connection Management:
Each request gets a dedicated connection via `get_db_connection()`. Connection is closed in finally block to prevent leaks. This works for request-based processing but doesn't scale to millions of concurrent connections.

Agent Polling Transactions:
Agents use optimistic locking to claim work:

- **Step 1:** SELECT FROM fraud_alerts WHERE pra_processed = 0 LIMIT 1
- **Step 2:** UPDATE fraud_alerts SET pra_processed = 2 WHERE id = X (state="claimed")
- **Step 3:** PROCESS (may take seconds/minutes)
- **Step 4:** UPDATE fraud_alerts SET pra_processed = 1 (state="complete")

If Step 4 fails, Step 1 retry will pick up the alert again

The intermediate state (pra_processed = 2) prevents multiple agents from processing same alert.

Trade-offs

Pessimistic Locking vs. Optimistic Locking:
Current approach uses optimistic locking (CAS-like integer transitions: 0 → 2 → 1).

Optimistic:

- **Pros:** Scales better (no locks held), simpler (no deadlocks)
- **Cons:** Requires retry logic, sensitive to race conditions

Pessimistic (SELECT ... FOR UPDATE):

- **Pros:** Guaranteed exclusive access, no retries needed
- **Cons:** Locks held longer, higher latency, deadlock risk

For fraud detection where agent processing takes seconds or minutes, optimistic locking is correct. Pessimistic would be deadlock-prone.

Implicit vs. Explicit Transactions:
Current approach: explicit BEGIN/COMMIT/ROLLBACK. Alternative: autocommit mode where each statement is auto-committed. Explicit is better for data integrity but requires careful coding to avoid leaving transactions open.

Read Committed vs. Repeatable Read:
Could use READ_COMMITTED isolation (faster, fewer locks) but risk dirty reads. REPEATABLE_READ prevents anomalies at cost of slightly higher latency. For fraud detection, consistency is paramount.

Transaction Rollback on Error:
Currently, if an exception occurs during transaction, rollback must be explicit in exception handler or finally block. Alternatively, use decorator pattern (@transactional) to handle rollback automatically. Currently, this isn't implemented uniformly.

Alternative Solutions Considered

NoSQL (MongoDB) Transactions:
MongoDB supports ACID transactions across multiple documents (v4.0+). Could reduce complexity of relational joins but loses proven reliability.

Event Sourcing:
Store events (FraudAlertCreated, FraudAlertProcessedByPRA) rather than current state. Provides audit trail but adds complexity (event replay logic, consistency verification).

Distributed Transactions (XA Protocol):
For multi-database scenarios. Not applicable for single MySQL instance but would be needed for multi-shard deployments.

SQLAlchemy ORM:
Provides transaction abstraction layer. Could simplify DAO code and handle rollback automatically. Trade-off: abstraction overhead, less control over SQL.

5.7 Other Applicable Technical Services

Initialization & Startup

Strategy:
The `ensure_tables_exist()` function runs at application startup, verifying all required tables and columns exist. This idempotent initialization approach enables seamless deployment without manual database setup steps.

Approach:
Rather than separate SQL migration files, schema definitions are embedded in Python. This couples code versioning with schema versioning but simplifies deployment for monolithic applications. For distributed team deployments, separate migration tools (Alembic, Flyway) would be preferable.

- **Trade-off:** Embedded schema definitions are simpler to maintain but don't scale to complex multi-version deployments. Current approach is acceptable given project scope.
- **Alternative:** Use Alembic (Python migration tool) for more sophisticated version control and rollback capabilities.

Caching Strategy

Current State: No explicit caching is implemented. Performance-critical queries could benefit

Trade-off: Simple but loses accuracy if data changes mid-request. Appropriate for read-only reference data but risky for frequently-changing fraud data.

Redis Distributed Cache:
For multi-instance deployments, Redis provides shared cache:
Code:

```
import redis
cache = redis.Redis(host='redis', port=6379)
profile = cache.get(f"profile:{customer_id}")
```

Trade-off: Additional infrastructure (Redis server) but enables shared cache across multiple app instances. Necessary for horizontal scaling.

Database Query Cache (Query Result Tuples):
Cache SELECT results with TTL. MySQL query cache is deprecated; application-level caching is the current approach.

Strategy Recommendation:

- In-memory cache for: customer profiles (read rarely), fraud pattern typologies (read-only reference)
- Redis cache for: recent transaction scores (read/write frequent), OTP tokens (time-sensitive)
- No cache for: fraud_alerts, payment_transactions (must be up-to-date)

Validation Framework

Current State: Validation is implicit in service layer logic. No centralized validation framework.

Recommended Strategy:
Implement schema-based validation at multiple levels:

Input Validation (API Gateway/Routes):
Validate request bodies before passing to service layer:
Code:

```
from pydantic import BaseModel, validator
class PaymentRequest(BaseModel):
    sender_account: str
    recipient_account: str
    amount: float

    @validator('a mount')
    def amount_positive(cls, v):
    if v <= 0:
        raise ValueError('Amount must be positive')
    return v
```

Business Logic Validation (Service Layer):
Validate business rules (customer can't transfer more than daily limit):
Code:

```
def transfer(sender_id, amount):
    daily_transferred = get_daily_transfer_total(sender_id)
    if daily_transferred + amount > DAILY_LIMIT:
        raise ValidationError("Daily limit exceeded")
```

Database Constraints (Persistence Layer):
NOT NULL, UNIQUE, FOREIGN KEY constraints prevent invalid data at storage.

Trade-off: Multi-layer validation is redundant but provides defense-in-depth. Could be consolidated into single layer but risks data corruption if validation is bypassed.

Alternative: Use declarative validation libraries (Marshmallow, Cerberus) instead of custom logic.

Fault Tolerance & Resilience

Current Implementation:

Agent Fault Tolerance:
Stale claim recovery (pra_processed reset after 120s) handles agent crashes. When agent crashes mid-operation, recovery automatically resets work to allow retry.

Database Unavailability:
`get_db_connection()` returns None on connection failure. Services gracefully degrade, though with reduced functionality.

Secrets Management:
Passwords, API keys should come from secrets manager:

- Development: .env file (never committed to git)
- Production: AWS Secrets Manager, Kubernetes Secrets, HashiCorp Vault

Configuration Tiers:

- Application defaults (safe values)
- Environment-specific (test/staging/prod)
- Runtime overrides (command-line flags, API)

Trade-off: More complex deployment but essential for security and multi-environment management.

6 Risks/Limitation

The Jatayu fraud detection system, while architecturally sound for mid-scale deployment, faces several critical constraints that impact production readiness and scalability. This section identifies key risks across three dimensions: technical performance ceilings that limit throughput, operational resilience gaps in infrastructure reliability, and security compliance deficiencies that prevent regulatory certification. Understanding these limitations is essential for determining appropriate deployment scope and identifying mandatory improvements before production traffic exposure. The assessment categorizes risks by severity and phase to guide remediation priorities and risk acceptance decisions.

The Phase 2 carries three critical production blockers:

(1) Scalability ceiling at 50 TPS—single Flask worker in app.py causes fraud_alerts queue buildup; PRA polling (500ms interval) cannot drain faster than 2K alerts/hour, resulting in 30+ minute detection latency.
(2) Operational downtime risk—EC2 hardware is not present, therefore if integrated, and if failure triggers 30+ minute manual recovery (no RDS Multi-AZ, no auto-failover), unacceptable for ongoing transactions.
(3) Security gaps—unencrypted EBS (ChromaDB), no field-level encryption, missing MFA, incomplete audit trails cause **PCI-DSS audit failure**.

Medium-level risks include BiLSTM model drift (no retraining pipeline), KB staleness (3-month lag in SWIFT fraud feed ingestion), and integration brittleness (gateway timeout blocks TMA).

Phase 2 acceptable only for controlled pilot (max 30 TPS, 95% uptime SLA).

Phase 3 mandatory blockers: distributed workers (100+ TPS), multi-AZ RDS failover, full encryption, automated retraining, event-driven coordination replacing polling.

Monitor KPIs: fraud_alerts.pra_processed=0 queue depth (alert if > 5K), TPS sustained (red line 40+), BiLSTM accuracy (monthly), KB age (warn if > 60 days). Production approval requires Phase 3 completion; Phase 2 MVP-only with explicit risk sign-off.

7 Alternative Solutions Considered

This section documents the major architectural alternatives evaluated during the design of the Eagle Trust Financial Services Platform and explains the rationale for the selected solution.

7.1 Major Architectural Alternatives

Alternative 1: Monolithic vs. Distributed Multi-Agent Architecture

- **Selected: Multi-Agent Distributed | Alternative: Monolithic:** The monolithic approach would run all components (API routes, services, agents) in a single Flask process sequentially. The distributed approach separates agents into independent processes that coordinate via database-backed work queues.
- **Monolithic Advantages**: Simpler deployment, easier debugging, lower infrastructure costs.
- **Monolithic Disadvantages:** Slow Pattern Recognition Agent blocks real-time blocking (ABA latency increases). No independent scaling. Single point of failure. Resource contention between high-memory agents and API handlers.
- **Why Distributed Selected**: Fraud detection requires <100ms blocking latency. Sequential execution violates this. If PRA takes 1 second, ABA is delayed 1 second. Independent agents with database coordination enables parallel processing: PRA analyzes while RAA synthesizes while ABA blocks—all simultaneously.

Alternative 2: Persistence: MySQL vs. PostgreSQL vs. NoSQL

- **Selected: MySQL | Alternatives: PostgreSQL | MongoDB:** MySQL provides relational schema with JSON column support. PostgreSQL offers superior JSON querying (JSONB). MongoDB offers schema flexibility.
- **PostgreSQL:** Would be a drop-in replacement (SQL dialect differences minimal in DAOs). Preferred for complex JSON querying, but the JSON usage is simple (boolean flags, ID arrays). PostgreSQL overhead unjustified.
- **MongoDB:** Abandons ACID guarantees, which are non-negotiable for financial transactions. Lost foreign key constraints. Regulatory compliance expects relational consistency.
- Why MySQL Selected: Highly relational schema. ACID guarantees mandatory for financial systems. No complex JSON querying. Lower overhead for this project's scale.

Alternative 3: Inter-Process Communication: Database Polling vs. Kafka vs. gRPC

- **Selected: Database Polling | Alternatives: Kafka | RabbitMQ | gRP:** Database-backed polling via state transitions (0→2→1) in fraud_alerts table. Agents poll every 100-500ms, claim work, process, signal completion.

- **Kafka Advantages:** High-throughput (millions/sec), preserves message ordering, industry standard. Disadvantages: Operational complexity (cluster management), network latency, overkill for current volume.
- **RabbitMQ Advantages**: Simpler than Kafka. Disadvantages: Still requires external infrastructure, messages deleted after consumption (bad for audit compliance), less suitable for fraud decision audit trails.
- **gRPC Advantages**: Low latency (<1ms direct calls). Disadvantages: Tight coupling (agents must know about each other), blocking calls (slow agent delays others), no audit trail, cascading failures (ABA down blocks RAA).

Why Database Polling Selected:

- ACID guarantees (either work succeeds completely or rolls back completely)
- Audit compliance (work items persisted forever for investigation)
- Fault tolerance (stale claims reset after 120s handles crashes gracefully)
- Simplicity (no external infrastructure, schema-based coordination)
- When throughput demands exceed this (1,000s of fraud alerts/sec), Kafka migration path is clear (agents consume from Kafka topics instead of polling tables)

Trade-off: Polling introduces latency jitter (0-500ms between poll cycles). Acceptable for fraud detection; blocking a transaction 200-700ms vs. 150-200ms is immaterial compared to missing the block entirely.

Alternative 4: Frontend: React SPA vs. Server-Rendered vs. Minimal JS

- **Selected: React SPA | Alternatives: Server-Rendered | Minimal J:** React single-page application with client-side routing and real-time state updates.
- **Server-Rendered (Flask + Jinja2):** Each page reloads from server. Disadvantages: Full page reload on every interaction is poor UX. Can't update fraud dashboard without refresh. Requires WebSocket anyway for real-time, negating simplicity advantage.
- **Minimal JS (hand-written HTML + fetch):** Disadvantages: All interaction reloads page. No state management (unmaintainable as app grows). Impossible to reuse components. HTML duplication everywhere.
- **Why React Selected:** Real-time dashboard requires fast client-side re-renders. React + Material-UI provides component reusability without over-engineering (compared to Angular).

Alternative 5: Agent System: Multi-Agent vs. Single Decision Engine

- **Selected: Multi-Agent | Alternative: Single Monolithic Function**
- **Single function integrating all fraud logic:** behavioral check → pattern check → regulatory check → synthesize decision.
- **Monolithic Advantages: Single point of control, easier to understand.**
- **Monolithic Disadvantages**: Adding new detection types requires modifying one function. ML model updates risk breaking existing logic. Resource contention (all processing in one function). Can't scale independent components. Can't update PRA without redeploying ABA.
- **Why Multi-Agent Selected:** Long-lived system requires flexibility. Can update PRA's BiLSTM model without touching ABA's blocking logic. New Regulatory Agent added in parallel without reordering. Each agent fails independently (PRA crash doesn't block ABA).

Alternative 6: Authentication: Sessions vs. JWT vs. SAML

- **Selected: Session-Based | Alternatives: JWT | SAM**
- Flask sessions stored on server with secure cookies (HTTPOnly, SameSite=Lax).
- JWT Advantages: Stateless (suitable for distributed services). Disadvantages: No instant revocation. If customer account frozen due to fraud, JWT valid for 8 hours. Unacceptable for fraud action (block customer now, not later).
- SAML Advantages: Enterprise SSO standard, centralized identity (Active Directory). Disadvantages: Overkill for customer authentication. Designed for intra-enterprise, not customer-facing.
- **Why Sessions Selected:** Instant revocation essential. Customer frozen → session cleared → next request fails 401. JWT can't match this behavior within token lifetime.

Alternative 7: Deployment: Docker Compose vs. Kubernetes vs. VMs

- **Selected: Docker Compose | Alternatives: Kubernetes | VMs**
- Containerized app via Docker Compose for orchestration.
- **Kubernetes Advantages:** Production-grade orchestration, auto-scaling, service discovery. Disadvantages: Steep learning curve, high operational overhead, Kubernetes tax on infrastructure, overkill for single-instance deployments.
- **VMs Advantages:** Familiar to traditional ops. Disadvantages: Slow startup (30s+), large resource footprint, configuration drift.
- **Why Docker Compose Selected:** Current deployment is single app instance + database. Kubernetes overhead unjustified. When needing multiple agent workers, Kubernetes becomes attractive (auto-scale PRA from 1 to 3 replicas based on queue depth).

Alternative 8: Schema Evolution: Current vs. Event Sourcing

- Selected: Current State + Audit Columns | Alternative: Event Sourcing
- Store `pra_processed`, `pra_verdict` directly in fraud_alerts (current state). Separate `aba_execution_log` for audit.
- **Event Sourcing:** Immutable events (FraudAlertCreated, PRAProcessed, ABAExecuted) persisted forever. State reconstructed by replaying events.
- **Event Sourcing Advantages:** Complete temporal history, state at any point in time, debugging simplified.
- Event Sourcing Disadvantages: Increased storage, slower reads (reconstruct from events), complex implementation (replay logic, snapshots), harder debugging (event semantics).
- Why Current Selected: Current approach provides 90% of event sourcing benefits (audit_execution_log captures decisions, fraud_alerts tracks state transitions) with 10% of complexity. If business requires "What was risk_score on March 15?" temporal queries, event sourcing becomes valuable.

7.2 Evolution Paths

The architecture is designed for evolution:

- **Scale Path:** Database polling (current) → Kafka (millions TPS) by replacing DAO layer. Agent logic unchanged.
- **Regulation Path:** Add new agents (GDPR compliance agent, cross-border reporting agent) without modifying existing agents.
- **ML Path:** Swap BiLSTM for transformer models in PRA. Agent interface unchanged.
- **Infrastructure Path:** Docker Compose → Kubernetes as worker count scales.

Appendix

The Autonomous fraud detection architecture is supported by comprehensive data models, database schemas, and operational infrastructure that enable real-time fraud detection and regulatory compliance. The system maintains 25 interconnected database tables including `fraud_alerts` (central coordination point), `action_packages` (ABA work queue), `customer_behaviour_profiles` (behavioral baseline), `pattern_alerts` (PRA outputs), and `cla_archive` (regulatory filings) that collectively capture the complete lifecycle of fraud transactions from detection through resolution. Critical design elements include idempotent schema migrations via the `_safe_alter()` function for zero-downtime deployments, a recovery mechanism that automatically resets stale agent claims after 120 seconds to prevent transaction loss from agent crashes, and compound database indexes (`idx_pra`, `idx_raa`, `idx_aba`) that enable sub-2ms polling queries across millions of records. The system uses a state machine pattern (0=pending, 2=claimed, 1=complete) for distributed coordination without message brokers, integrates ChromaDB vector database for RAG-based pattern matching, and includes fallback mechanisms for cold-start customers, RAG unavailability, and Redis cache failures, ensuring fraud detection continues reliably even during partial system degradation.

1. Expected Software Response

Transaction Processing Flow:

- Customer initiates transaction → transactions table created
- Monitoring Agent (MA) evaluates behavioral anomaly within 100ms → populates risk_score
- Pattern Recognition Agent (PRA) analyzes sequence patterns within 500ms → populates pattern_score, bilstm_score
- Regulatory Assessment Agent (RAA) applies compliance rules within 500ms → populates raa_verdict (ALLOW, REVIEW, FLAG, BLOCK)
- Alert & Block Agent (ABA) executes gateway actions within 100ms → updates customers.is_frozen if required
- Compliance Logging Agent (CLA) generates CTR/STR filings as needed.

Expected Outcomes:

- **ALLOW Decision:** Transaction processed, no alert created. SLA: <50ms response time.
- **FLAG Decision:** Transaction processed with warning, fraud_alert created with LOW priority. Customer notified. SLA: <200ms.
- **BLOCK Decision:** Transaction rejected, account frozen, fraud_alert created with HIGH priority, immediate otp_tokens invalidation. SLA: <150ms.
- **ALERT Decision:** Transaction flagged for regulatory review, filing queued. SLA: <300ms.

Error Handling Response:

- **RAG Unavailable:** PRA falls back to ML-only scoring (no L1/L2 adjustments).
- **Cold Start Profile:** New customers use conservative thresholds until 30 transactions accumulated.
- **Database Timeout:** Agents retry with exponential backoff (100ms → 200ms → 500ms); fallback to in-memory queue.

2. Performance Bounds

Latency Requirements:

Component	SLA	Benchmark	Notes
MA Evaluation	<100ms	Current: 45ms	Real-time anomaly scores
PRA BiLSTM	<250ms	Current: 120ms	Model inference on GPU
RAA Rule Engine	<200ms	Current: 80ms	Regulatory logic + CTR lookup
ABA Execution	<100ms	Current: 40ms	Gateway call + account freeze
Full Pipeline (E2E)	<750ms	Current: ~320ms	95th percentile latency

Throughput Requirements:

- Peak Load: 2,000 transactions/minute (33 txn/sec)
- Sustained Load: 1,000 transactions/minute during business hours
- Database Polling Overhead: 5 SELECT queries/second per agent × 5 agents = 25 QPS (compound indexes ensure <2ms per query)

Database Capacity:

- fraud_alerts table: 100M rows = ~150GB (15 months of transactions at 2M/day volume)
- Retention policy: Archive rows >180 days to cla_archive table
- Index maintenance: Nightly ANALYZE TABLE; quarterly OPTIMIZE TABLE

Memory Footprint:

- Python Flask app: ~200MB (base) + 50MB per agent = ~450MB for 5 agents
- BiLSTM model (bilstm_v1.pt): ~85MB (loaded once at startup, shared)
- ChromaDB vector store (in-memory cache): ~200MB (top 10K embeddings for L2/L3)
- Total per container: ~750MB allocations

3. Identification of Critical Components

Tier 1 - System-Critical (Failure = Complete Outage):

- **MySQL Database (customers, fraud_alerts tables)**
  - Single point of persistence; all agent state depends on these tables.
  - Mitigation: Daily backups, replication to standby, automated failover.
  - Recovery Time Objective (RTO): <5 minutes.

- **PRA BiLSTM Model (bilstm_v1.pt)**
  - Only source of pattern detection scoring; other agents depend on PRA verdict.
  - Mitigation: Model versioning, A/B testing of new versions, fallback to statistical baseline.
  - Critical Path: Model must load successfully at agent startup; 30s timeout triggers fallback scorer.

- **ChromaDB Vector Database (L1/L2/L3 citations)**
  - RAG layer; missing citations degrade PRA scoring accuracy by ~15%.
  - Mitigation: In-memory cache of top embeddings, graceful degradation if unavailable.
  - Performance Impact: Query time >5s triggers fallback to keyword-based retrieval.

Tier 2 - Service-Critical (Failure = Degraded Performance):

- **ABA Gateway Controller**
  - Responsible for account freeze/unfreeze commands; delayed execution risks fraud leakage.
  - Mitigation: Retry logic (3 attempts), fallback to manual operator queue.
  - Critical Threshold: If >10% gateway calls fail, escalate to ops team.

- **State Machine Recovery Logic (_safe_alter, claim timeout 120s)**
  - Prevents stale alerts from blocking pipeline; recovery failures cascade.
  - Mitigation: Separate recovery cron job runs every 5 minutes as safety net.
  - Critical Threshold: If recovery finds >100 stale claims, investigate agent crashes.

- **CLA PDF Generator & Archiver**
  - Regulatory compliance requirement; missing filings create audit risk.
  - Mitigation: Async processing with retry queue, alert on filing failures.
  - Critical Threshold: CTR/STR filing SLA breach triggers compliance team notification.

Tier 3 - Optional (Failure = Feature Loss, Not Outage):

- OTP Fallback Columns (customers.otp_attempt_count, soft_lock_until)
  - Used when Redis cache unavailable; degrades to database-backed rate limiting.
  - Impact: ~10% performance overhead during Redis outages.

- Notification Queue (notification_queue table)
  - Async customer alerts; failures don't block transaction processing.
  - Impact: Customers notified 30s later instead of real-time.

4. Review Comments on Architectural POC

POC Scope: Proof-of-concept implemented with single Monitoring Agent + simplified rule engine. Validated core concepts: database-backed work queue, polling-based agent coordination, recovery mechanism.
Strengths Validated:

- Database Polling Pattern Works at Scale: 500ms polling interval + compound indexes achieve <2ms latency even with 10M+ fraud_alerts rows.
- Recovery Mechanism Effective: 120s stale claim timeout successfully recovered crashed agents in test scenarios; no alert loss observed.
- Idempotent Schema Migrations Reliable: _safe_alter() pattern enables safe rolling deployments; tested with 50+ schema iterations.
- Multi-Agent Coordination via State Machine: Distributed coordination (0→2→1 transitions) avoided deadlocks; no race conditions detected under concurrent polling.

Weakness Identified & Addressed:

Issue	POC Finding	Resolution	Status
BiLSTM Model Cold Start	Model loading took 8s; agents blocked.	Async model loading at startup; non-blocking fallback enabled.	Fixed
RAG Latency	ChromaDB queries exceeded 2s at scale	In-memory embedding cache added; reduced to <200ms	Fixed
Database Connection	Exhausted connections with 5 concurrent agents.	Switched to persistent connections with health checks.	Fixed
JSON Column Updates	Fraud_alerts.anamoly_flags merge operations slow	Pre-allocate JSON structure; partial updates instead of Full merge	Fixed

Architectural Improvements Post-POC:

- Added compound indexes (idx_pra, idx_raa, idx_aba) → 40% latency reduction.
- Introduced action_packages table for ABA work queue → decoupled RAA from ABA, enabling independent scaling.
- Implemented novel_pattern_candidates staging table → feedback loop for PRA model retraining.
- Added aba_execution_log immutable audit trail → compliance requirement, investigation support.

Recommended Testing Focus for Production:

- Failover Testing: Kill agents gracefully; verify stale claim recovery.
- Load Testing: 2,000 txn/min for 24h; monitor latency tail (p95, p99).
- Data Consistency: Concurrent agent writes to same fraud_alert row; verify no mutations lost.
- Model Retraining: Validate BiLSTM retraining pipeline integrates with novel_pattern_candidates table.

5. Justification of Changes to Existing Architecture (Optional)

N/A — This is a greenfield fraud detection system; no existing architecture exists to modify.

However, if migrating from legacy monolithic system, the following changes justify the new distributed multi-agent approach:

Legacy Monolith Challenge → New Architecture Solution:

Legacy Issue	New Design	Justification
Single synchronous rule engine latency (5+ sec)	5 independent async agents (50-500ms)	Parallel evaluation enable sub-second SLA
Tightly coupled auth/fraud/payment logic	Layered separation: Routes→Services→DAO→DB	Service-oriented teams can own components independently
No pattern detection; rules-only	BiLSTM + RAG + rule synthesis	Reduces false positives 30% catches novel patterns
Manual regulatory filing process	Auto CTR/STR generation + CLA review queue	Eliminates 2 – hour filing delays

No audit trail for blocked transactions	aba_execution_log captures every decision	Regulatory audit trail; dispute investigation support
Hard to test monoliths	Testable agent interfaces; DAO mock layer	Unit test coverage increased from 12% to 78%
Zero recovery from crashes	State machine + 120 timeout recovery	Eliminated transaction loss; zero-downtime restarts

Migration Strategy (if applicable):

- Phase 1: Deploy new agents in parallel; shadow-run (log decisions, don't execute)
- Phase 2: Gradually shift 10% → 50% → 100% of transactions to new system
- Phase 3: Retire legacy system; validate regulatory compliance during cutover
- Rollback Plan: Maintain legacy system for 30 days; automatic traffic switchback if error rates spike >2%